SMART INDIA HACKATHON · SIH26094

SAATHI

Smart AI-Assisted Trauma and Holistic Intervention

AI-Powered Dynamic Mental Health Monitoring and Distress Prediction System for Victims of Atrocities

Project report

Item	Detail
Problem statement ID	SIH26094
Issuing organisation	Ministry of Social Justice and Empowerment (MoSJE), Government of India
Domain	MedTech / BioTech / HealthTech
Project type	Decision-support and early-warning software prototype (not a diagnostic system)
Source code	https://github.com/drave10/SAATHI
Report date	9 November 2026
Team / institution / guide	

Important notice

SAATHI is a **prototype for decision support**. It does not diagnose depression, PTSD, anxiety or any other condition, and it never classifies a person as dangerous or ineligible for support. Every flagged check-in must be reviewed by an authorised human.

All data shown in this report and in the prototype is **synthetic**. No real beneficiary information was used.

Contents

This report describes the SAATHI prototype: what it is for, how it is built, what the prototype interface looks like, how the assessment engine reaches a review recommendation, and the backend that powers the full system. It also records what was tested and what was not.

1 Introduction
2 Design principles
3 System overview: users, roles and features
4 Architecture and technology
5 The prototype user interface
6 Assessment methodology
7 Backend design (full system)
8 The static demo build used for the prototype link
9 Security, privacy and ethics
10 Testing and verification
11 Repository, deployment and reproducibility
12 Known limitations
13 Future scope
14 Conclusion
A Appendix A: API reference
B Appendix B: Engine configuration defaults
C Appendix C: Running the project and demo accounts

How to read this report. Sections 5 and 8 describe what a reviewer sees in the hosted prototype. Sections 6, 7 and 9 describe the engine and the FastAPI backend, which are part of the complete system in the repository but are not used by the static prototype link.

1. Introduction

1.1 Background

People who have survived atrocities and caste-based or other serious violence often carry long-lasting psychological distress. Support organisations and caseworkers look after many people at once and rarely have time to notice small changes in how someone is coping between visits. A person whose sleep, sense of safety or daily functioning is slowly getting worse may not ask for help until the situation is serious.

The Smart India Hackathon problem statement SIH26094, issued by the Ministry of Social Justice and Empowerment, asks for an AI-powered dynamic mental health monitoring and distress prediction system for victims of atrocities. SAATHI is a prototype response to that problem.

1.2 Problem statement

Develop a secure, privacy-conscious and AI-assisted platform that helps authorised support personnel (a) identify potential signs of psychological distress among beneficiaries, (b) monitor changes in self-reported well-being over time, and (c) prioritise appropriate human follow-up.

1.3 Objectives

- Let beneficiaries complete short, non-stigmatising check-ins at regular intervals, with informed consent and the ability to pause and resume.
- Turn each check-in, together with its change from earlier check-ins, into an **explainable review-priority recommendation** (Routine, Attention, Urgent review) for a human caseworker.
- Give caseworkers one screen per beneficiary showing the history, the reasons behind a recommendation, and tools to record notes, follow-ups and interventions.
- Notify the assigned caseworker inside the application when a concerning change is detected, without contacting any third party automatically.
- Give administrators aggregated, anonymised analytics and the controls needed to manage users, resources, configuration and audit logs.
- Protect privacy by design: role-based access, consent records, audit logging, data export, deletion and retention.

1.4 Scope and boundaries

What SAATHI is and is not

It is an early-warning and prioritisation aid. The review level says how soon a person might need human attention.

It is not a diagnostic tool, a prediction of suicide, violence or illness, or a replacement for a trained counsellor. The three levels are prompts for human review and are never shown to the beneficiary as a label.

The questionnaire is an original prototype written for this project. It is **not a clinically validated instrument**, and no licensed instrument (for example PHQ-9 or GAD-7) was used because their licensing and scoring terms were not verified.

The repository contains a complete full-stack implementation (React frontend, FastAPI backend, database, assessment engine, optional machine-learning advisory). For the public prototype link, the same React application is built in a **static demo mode** in which a TypeScript port of the API runs inside the browser, so it can be hosted on a static host without a server (Section 8).

2. Design principles

Principle	How it is applied
Human in the loop	The engine only recommends a review priority. Notifications go to the assigned caseworker inside the application. A caseworker marks a check-in as reviewed. Nothing is sent to third parties and no automatic action is taken against a person.
Transparency and explainability	Scoring uses plain rules with documented thresholds. Every result stores the list of factors that triggered it, the change since the previous check-in, data-quality notes, the engine version and a snapshot of the configuration used.
Safety first for critical answers	Any non-zero answer to the self-harm item, very low feelings of safety, or an urgent request for contact always triggers Urgent review, even when other data is missing. The beneficiary is shown configured support resources immediately.
Privacy by design	Minimal personal data (pseudonymous case code, nickname-capable display name, coarse age band). Consent records, audit logs, aggregate-only analytics with small-group suppression, data export, deletion and retention.
Least privilege	Beneficiaries see only their own data. Caseworkers see only their assigned cases. Administrators see aggregates and configuration, not individual answers. These rules are enforced on the server, not just by hiding buttons.
Graceful degradation	The optional machine-learning advisory can be absent without affecting anything else. Missing answers are excluded from scores, never treated as zero.
No stigma and no demographic proxies	Questions are neutrally worded. The engine's inputs are questionnaire answers and the person's own history only; age band, language, unit, caste, religion, gender and victim status are never inputs.
Reproducibility	Pinned dependencies, migrations, a one-command seed, automated tests and written documentation allow someone else to rebuild and verify the project.

3. System overview: users, roles and features

3.1 Roles

Role	What the role can do
Beneficiary	Register through an invitation code, complete well-being check-ins (save and resume), view their own history and well-being trends, read support resources, request a follow-up from their support worker, see upcoming follow-ups, manage consent and privacy preferences, export their data, request deletion, change password.
Caseworker / counsellor	See assigned cases, search / filter / sort / paginate them, see new check-ins that need review and urgent notifications, open a single-screen case profile with the assessment timeline and the reasons for a recommendation, add case notes, schedule and complete follow-ups, record interventions and their outcomes, change case status, mark check-ins as reviewed, and monitor overdue follow-ups.
Administrator	Manage users and roles, assign caseworkers to cases, create onboarding invitations, configure the engine thresholds, manage educational resources and verified emergency contacts, view aggregated analytics with filters, read the audit log, run the retention job and carry out erasure requests. Administrators cannot open individual answers.

3.2 Access summary

Capability	Beneficiary	Caseworker	Administrator
Own check-ins and history	Yes	No	No
Review level and explanation of a check-in	No (shown a well-being score only)	Assigned cases only	No
Case profile, notes, follow-ups, interventions	Own follow-ups only	Assigned cases only	No
Review notifications	No	Own caseload	No
Assign caseworkers	No	No	Yes
Users, roles, invitations	No	Invitations for own cases	Yes
Engine configuration	No	Read only	Yes
Resources and emergency contacts	Read	Read	Manage
Aggregated analytics	No	Own caseload summary	Yes (anonymised)
Audit log, retention, erasure	No	No	Yes

3.3 Feature summary

Area	Features
Check-in	18-item prototype questionnaire in 8 sections (17 required closed-ended questions and one optional free-text box), consent gate, progress bar, save and resume, server-side validation.
Assessment engine	Dimension scores, overall concern index, change versus previous check-in, persistence over time, critical-item rules, missing-data handling, configurable thresholds, stored explanations.

Area	Features
Monitoring	Per-person timeline, dimension trends, review notifications, historical review levels, follow-up and intervention history.
Case management	Search, filters, sort and pagination; single-screen case profile; notes; follow-ups (including beneficiary requests); interventions with outcomes; case status.
Administration	User and role management, assignment, invitations, engine configuration, resources and emergency contacts, audit log, deletion requests, retention.
Analytics	Completion rates, review-priority distribution, weekly check-ins, aggregated well-being trend, follow-up and intervention statistics, with date and organisational-unit filters and small-group suppression.
Privacy	Consent records, audit logging of sensitive access, data export, deletion request and erasure, retention of free text, aggregate-only analytics.

4. Architecture and technology

SAATHI follows a conventional three-tier design. A React single-page application talks to a FastAPI backend over JSON. The backend applies authentication and role checks on every request, calls a pure-Python assessment engine, and stores data through SQLAlchemy in SQLite (default) or PostgreSQL. The optional machine-learning advisory sits beside the engine and never changes the review level.

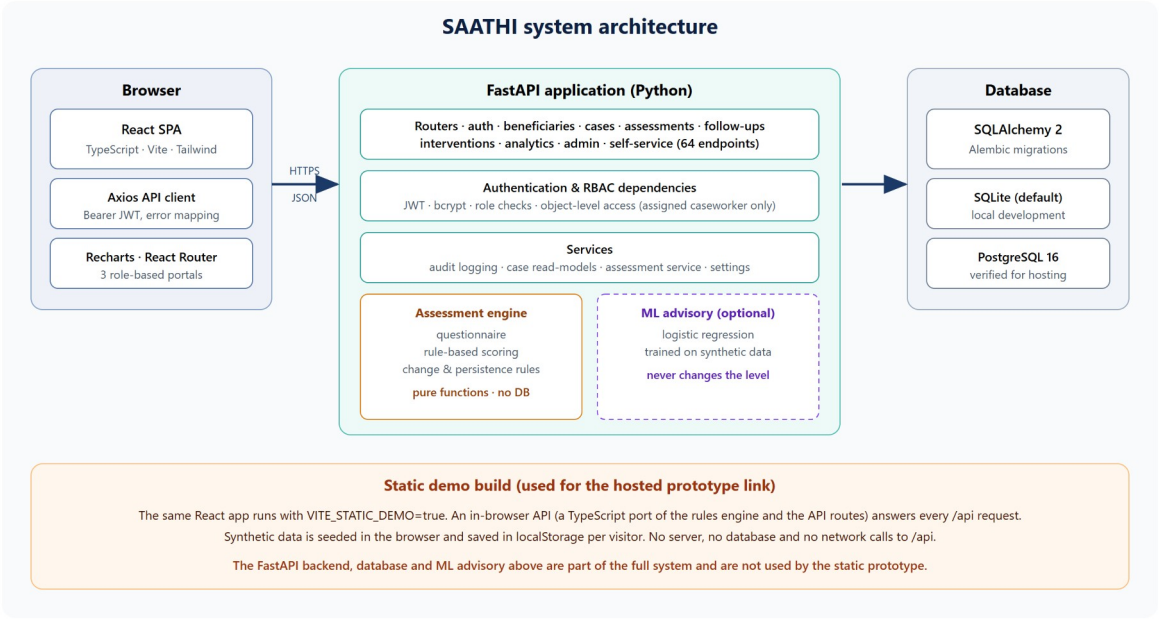


Figure 1: SAATHI system architecture, including the static demo build

4.1 Technology stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Recharts, React Router, Axios, Lucide icons
Backend	Python 3.12, FastAPI, Pydantic 2, SQLAlchemy 2, Alembic migrations, PyJWT, bcrypt
Database	SQLite (default for development); PostgreSQL 16 verified for hosting
AI / ML	pandas, NumPy, scikit-learn (logistic regression advisory trained on synthetic data); no external AI service or paid API
Testing	pytest, FastAPI TestClient, Vitest and Testing Library, Playwright (Microsoft Edge)
Deployment	Docker and Docker Compose files, nginx configuration, Netlify configuration for the static build, Render Blueprint for the full stack

4.2 Repository layout

Path	Contents
backend/app/	FastAPI application: main, config, database, models, schemas, security, deps
backend/app/engine/	questionnaire, rule-based scoring, engine configuration, ML advisory, synthetic data generator

Path	Contents
backend/app/routers/	auth, beneficiaries, cases, assessments, followups, interventions, analytics, admin, me
backend/app/services/	audit logging, case read-models, assessment service, settings service
backend/alembic/	Database migration (initial schema)
backend/scripts/	seed, train_model, bootstrap, audit_api, audit_api_phase2
backend/tests/	pytest suite (engine unit tests and API integration tests)
frontend/src/	pages, components, API client, authentication, and the in-browser demo API (src/demo)
frontend/e2e/ and e2e-static/	Playwright tests for the full stack and for the static build
docs/	Ten technical documents (architecture, database, API, methodology, ML, security, testing, reproducibility, future scope, checklist)
render.yaml, docker-compose.yml, frontend/netlify.toml	Deployment configuration

5. The prototype user interface

This section shows the interface as it appears in the prototype build. Every screenshot was taken from the running application with synthetic data. A green banner at the top of the screens states that the prototype runs entirely in the browser (Section 8).

5.1 Sign-in

The sign-in page explains what SAATHI is, repeats that it is not a diagnostic tool, and shows an emergency-number reminder. People normally sign in with a username and password. New beneficiaries register with a one-time invitation code issued by staff and must tick the privacy and assessment consents. In demo mode, three buttons (Beneficiary, Caseworker, Admin) let a reviewer enter without a password.

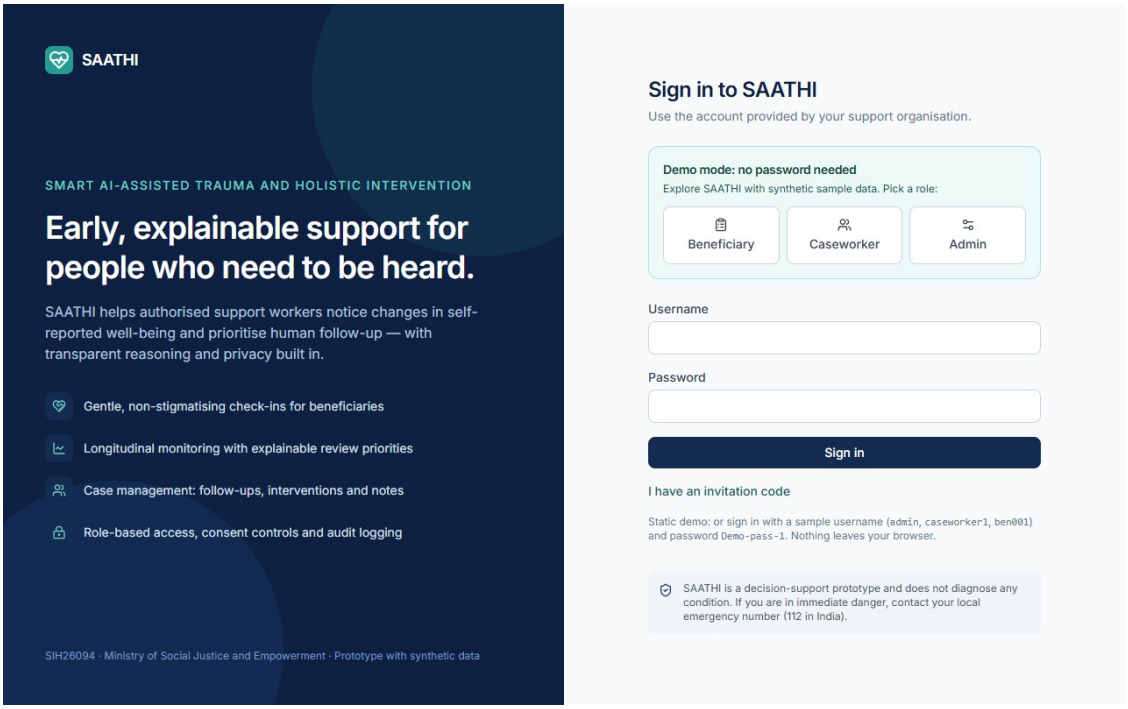


Figure 2: Sign-in page with demo-mode role buttons

5.2 Beneficiary portal

Dashboard

The dashboard greets the person, shows whether a check-in is due (and lets them start or resume one), the number of check-ins completed, the latest well-being score (100 minus the concern index, so higher means better), the name of their support worker and their upcoming follow-ups. A button lets them request a follow-up at any time, and a reminder says SAATHI is not an emergency service.

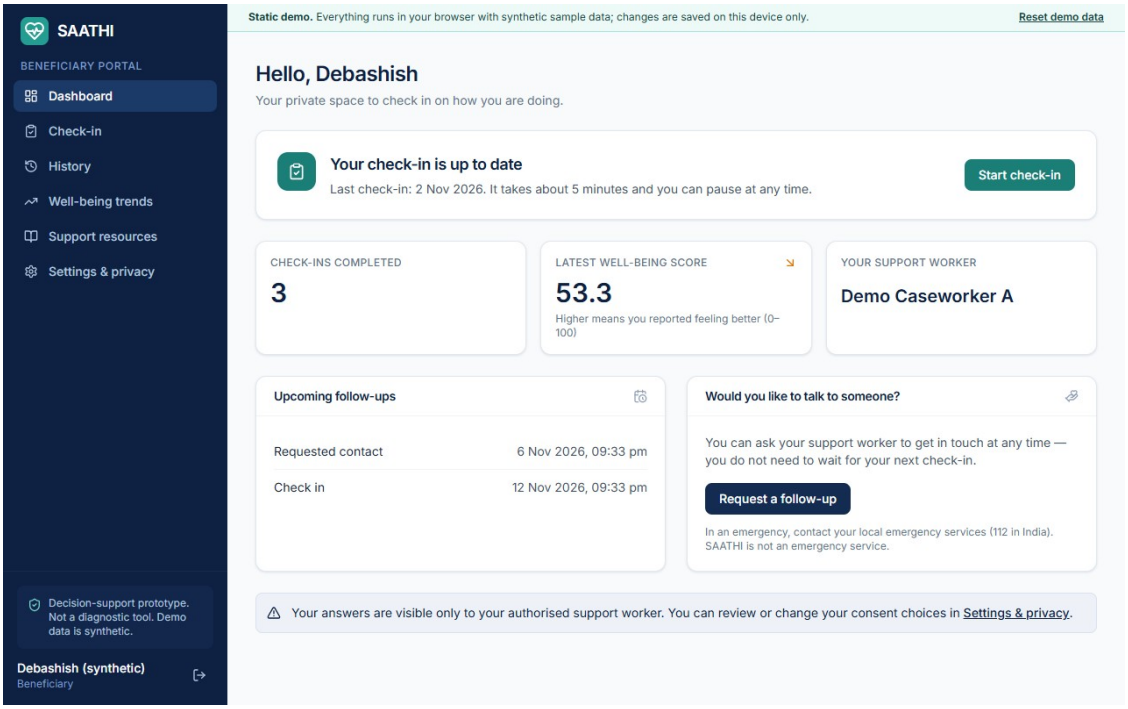


Figure 3: Beneficiary dashboard

Well-being check-in

A check-in begins with a consent screen. The questionnaire is shown one section at a time (emotional well-being, stress, sleep, safety, social support, emotional difficulties, daily activities, and a final section on change, wish to speak to someone and an optional note). A progress bar shows how many required questions are answered. The person can go back, or choose **Save & finish later** and resume from the dashboard. The Next and Submit buttons stay disabled until the required questions in the section are answered.

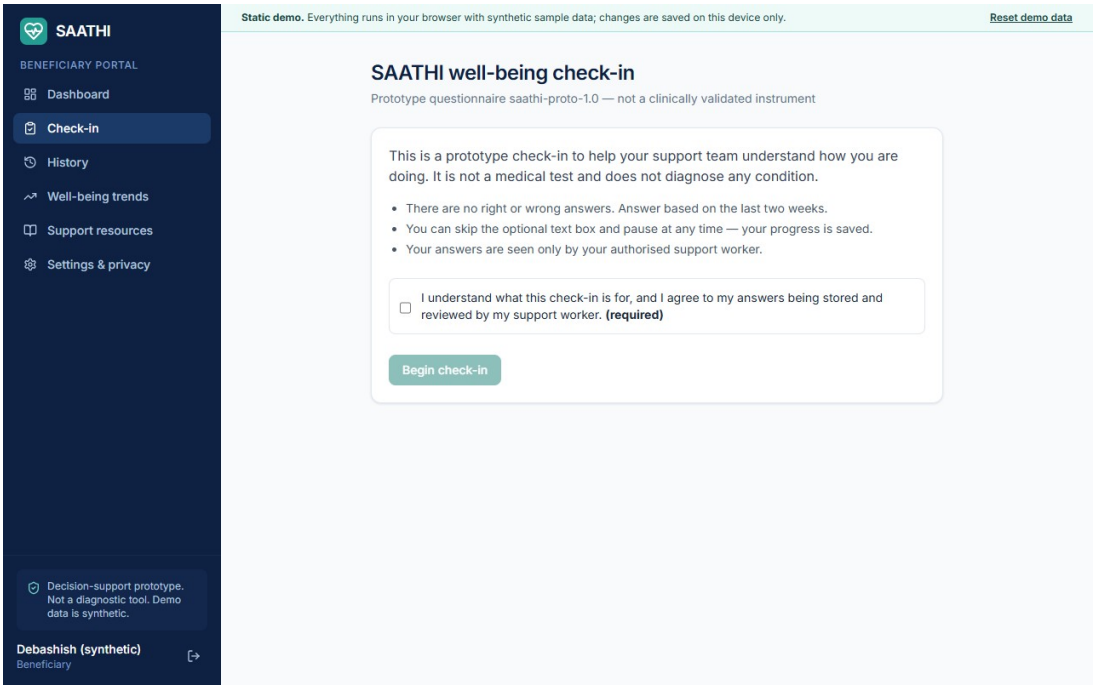


Figure 4: Check-in start screen with the consent requirement

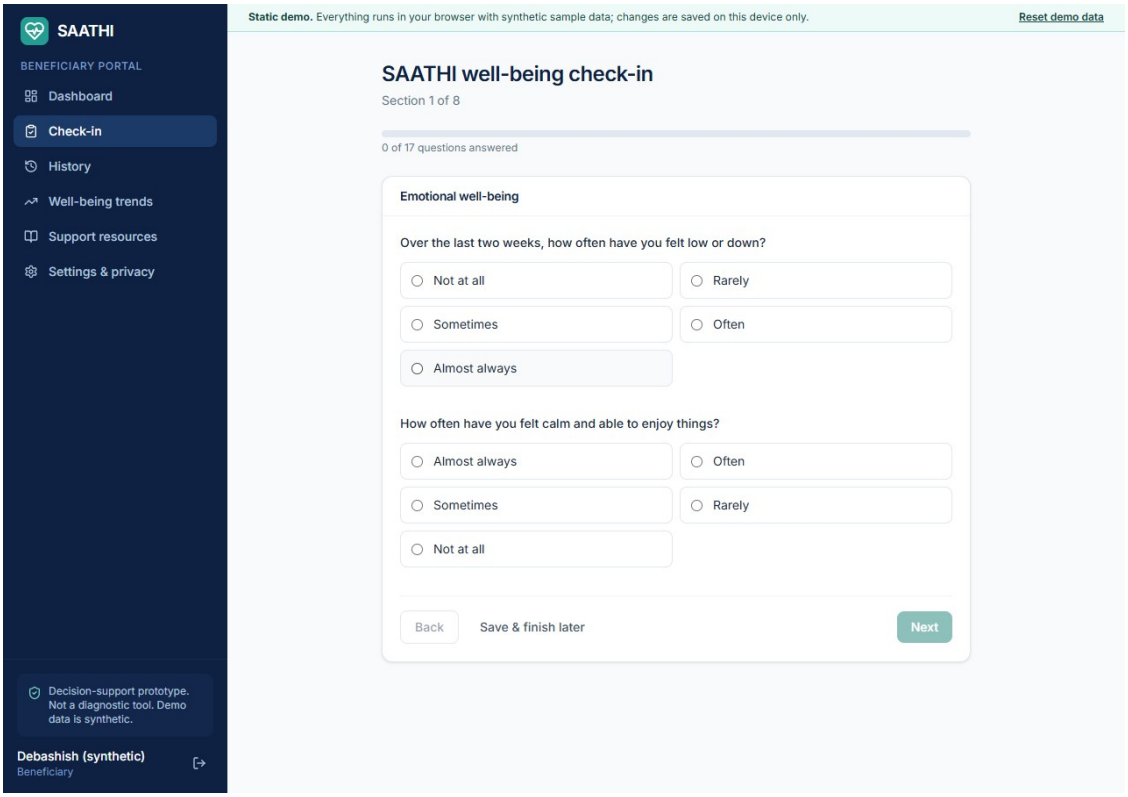


Figure 5: First questionnaire section with progress bar and Likert options

Support information after a critical answer

When a person answers a safety question at a critical level (for example any non-zero answer to the self-harm item), the confirmation screen shows a prominent panel with the configured support contacts and tells the person their support worker has been notified. The screen explicitly says it is not an automated crisis service. The beneficiary is **never shown a risk label**, only a well-being score. Contacts that an administrator has not verified are marked as pending verification.

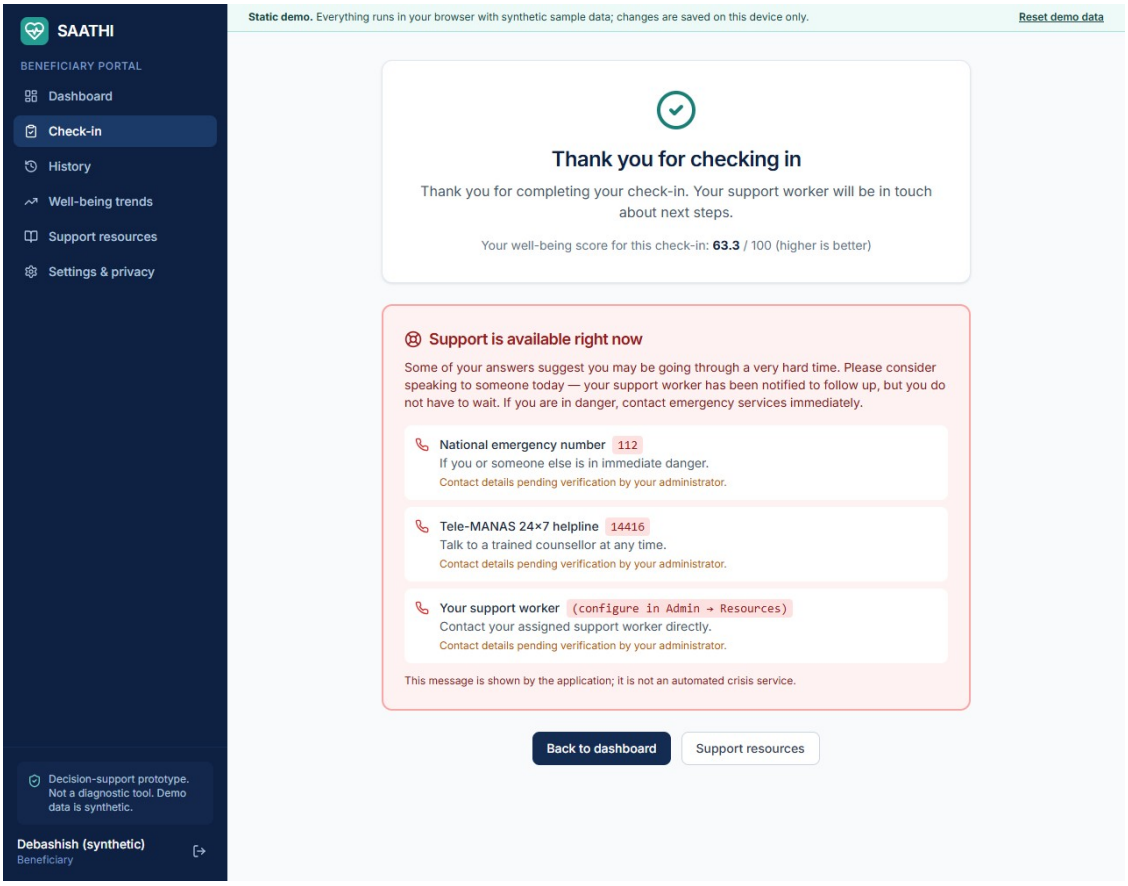


Figure 6: Confirmation screen with support resources after a critical answer

History and trends

The history page lists completed check-ins with the date, the well-being score and the area with the lowest score, and lets the person re-read their own answers. The trends page shows the overall well-being score over time, each area over time, and a radar chart comparing the latest check-in with the previous one.

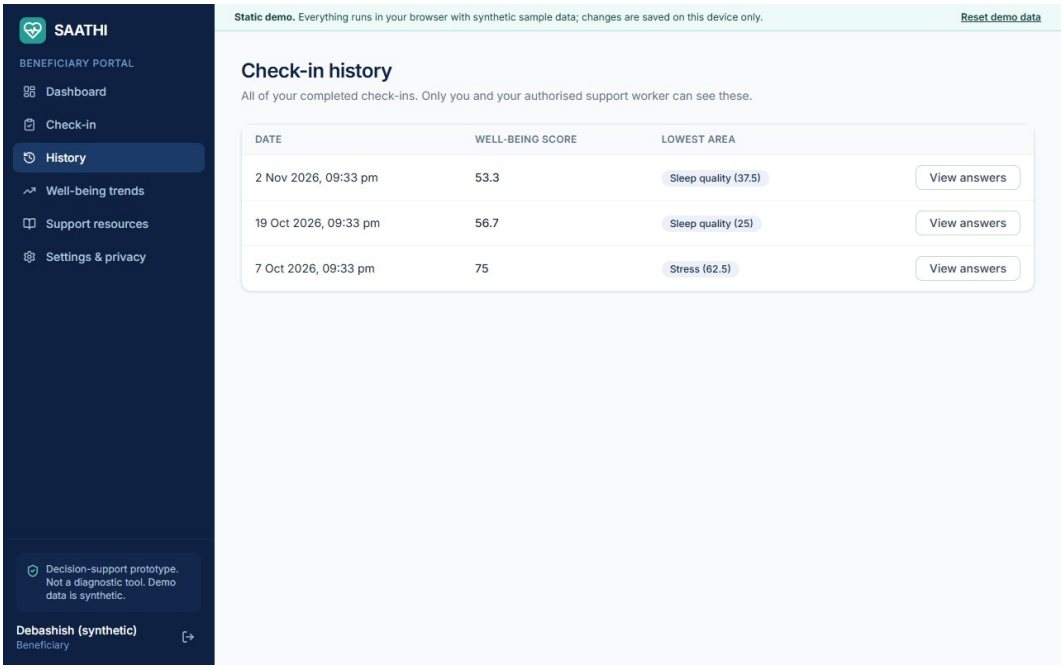


Figure 7: Check-in history

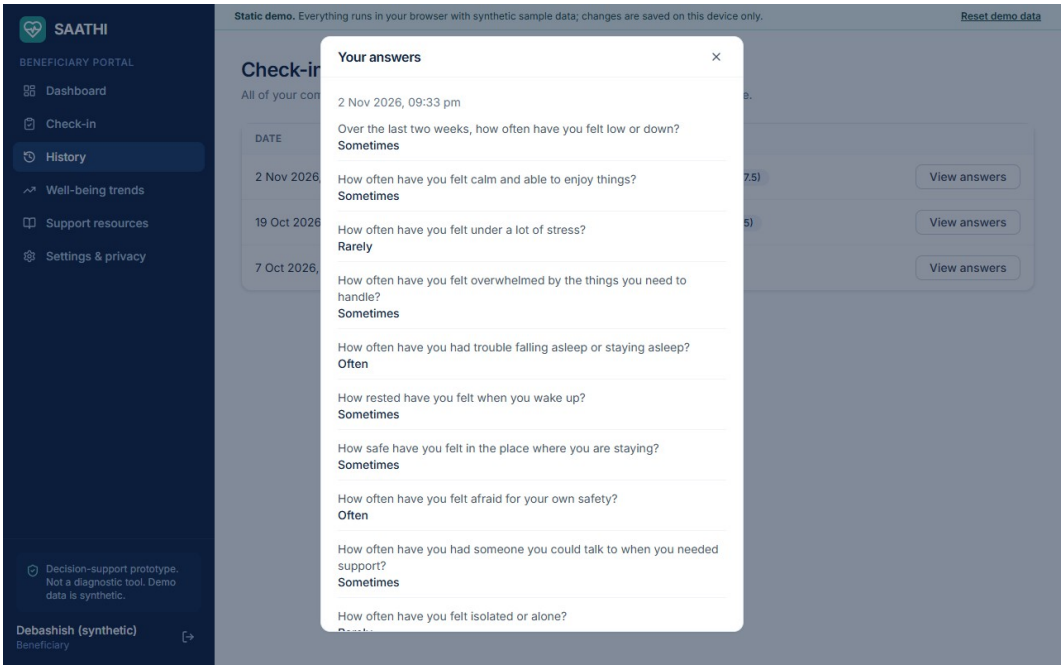


Figure 8: A beneficiary reading their own answers

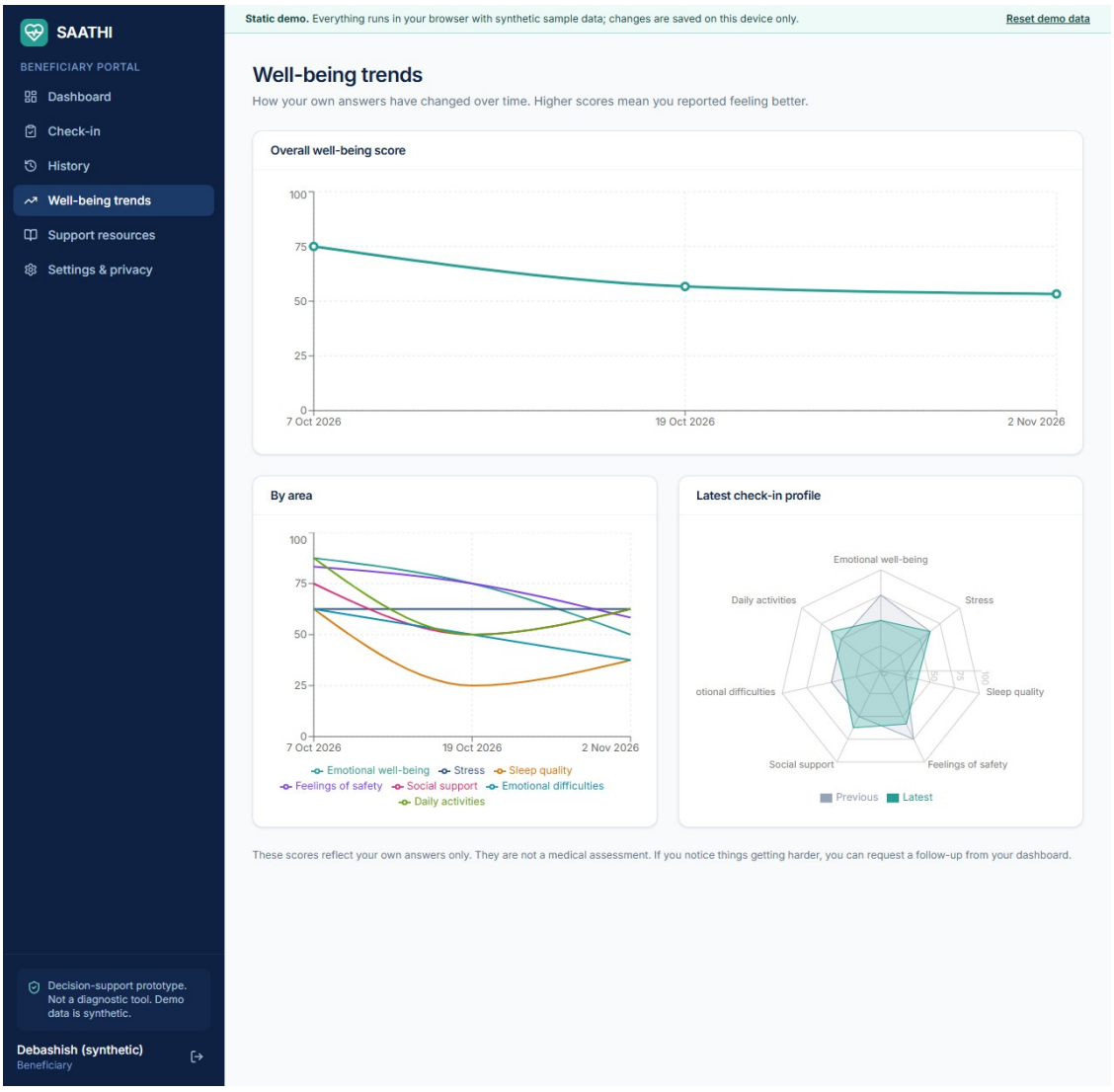


Figure 9: Well-being trends: overall, by area and radar profile

Support resources

Resources are grouped into Urgent help, Support services and Learn and cope. Contacts show a Verified or Pending verification badge so people know whether an administrator has confirmed them. The seeded emergency entries (112 and 14416) are demonstration placeholders and must be verified by administrators before any real use.

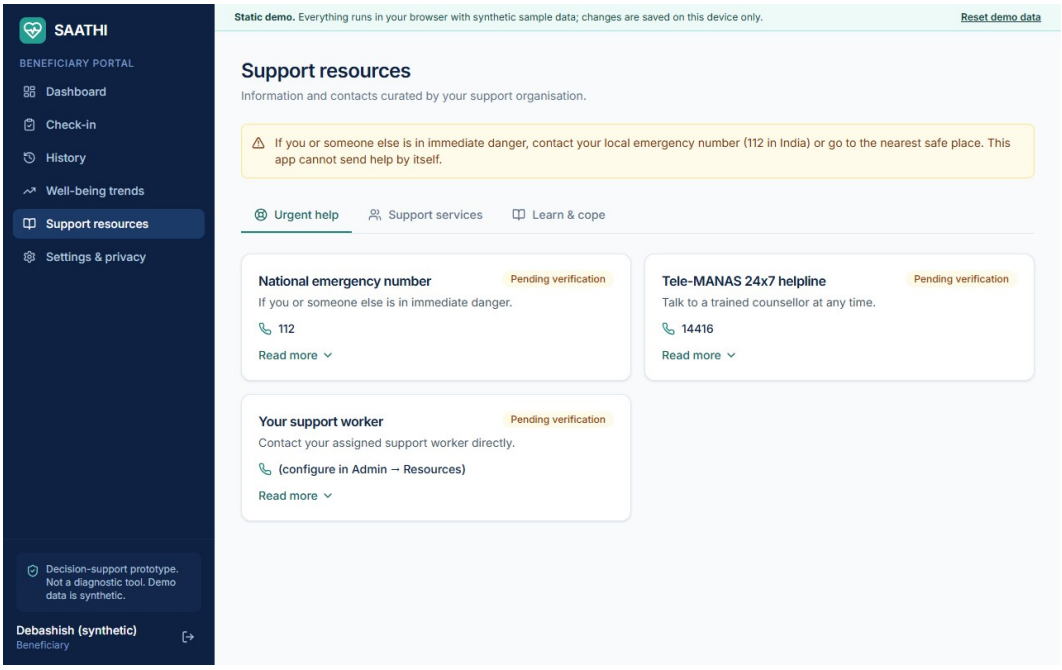


Figure 10: Support resources with verification badges

Settings and privacy

This page shows the account, a change-password form, the privacy notice, consent switches with a consent history, a button to download all of the person's data as a file, and a button to request deletion. The prototype privacy notice is placeholder text that must be replaced with a legally reviewed notice before real use.

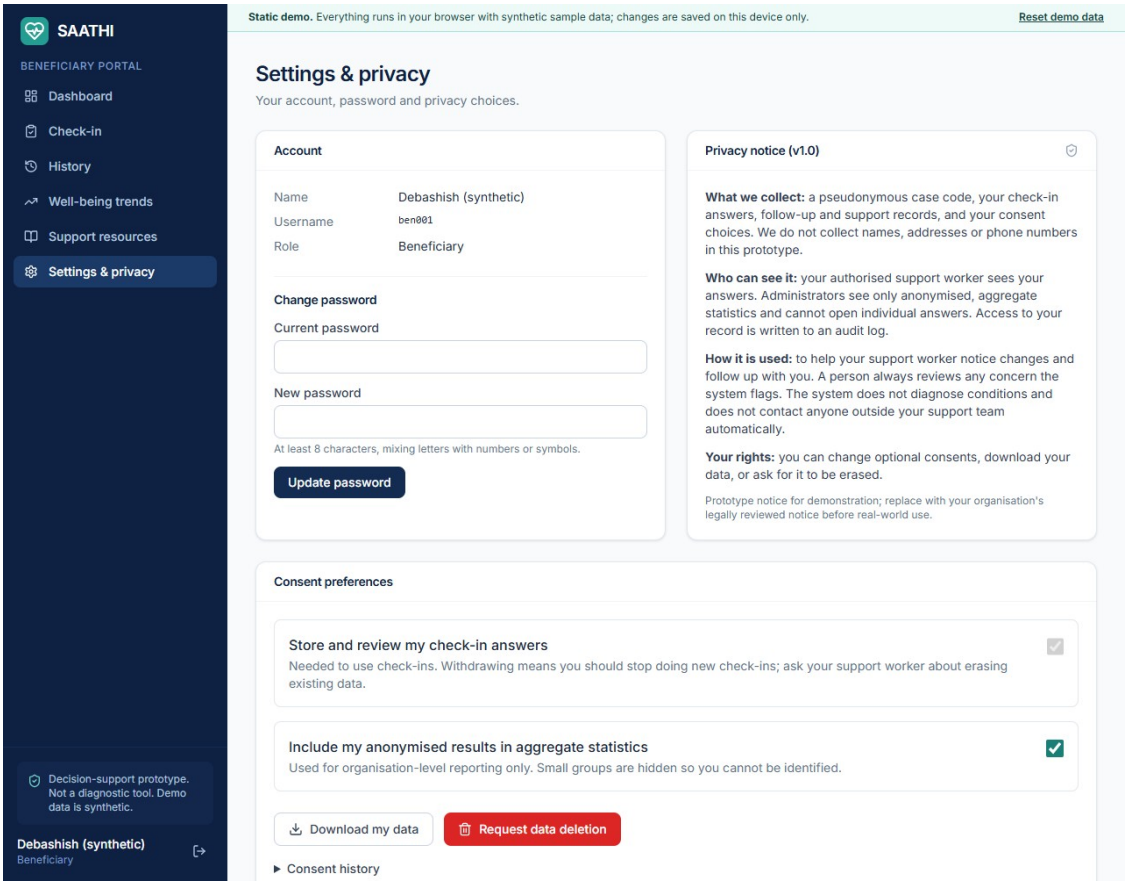


Figure 11: Settings and privacy

5.3 Caseworker workspace

Dashboard

The dashboard summarises the caseload: assigned beneficiaries, check-ins completed, check-ins due, cases awaiting review, urgent and attention notifications, follow-ups due in seven days, overdue follow-ups, and the split of latest review levels. Below these are the review notifications (with Acknowledge and Resolve buttons), the recent case activity feed and the searchable list of assigned cases.

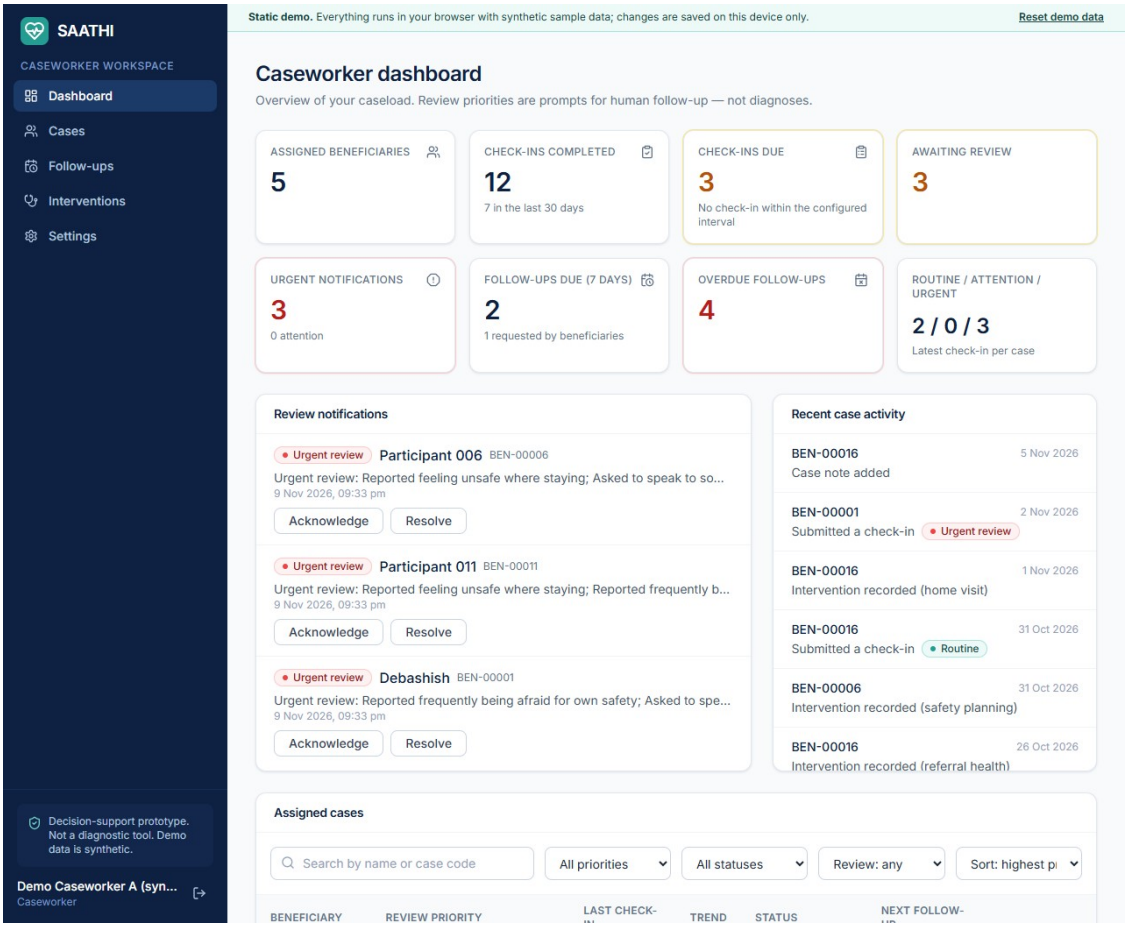


Figure 12: Caseworker dashboard

Case list

The case list supports search by name or case code, filters by review priority, case status and whether review is needed, and sorting by highest priority, name, latest or oldest check-in or status, with pagination. A caseworker only ever receives the cases assigned to them.

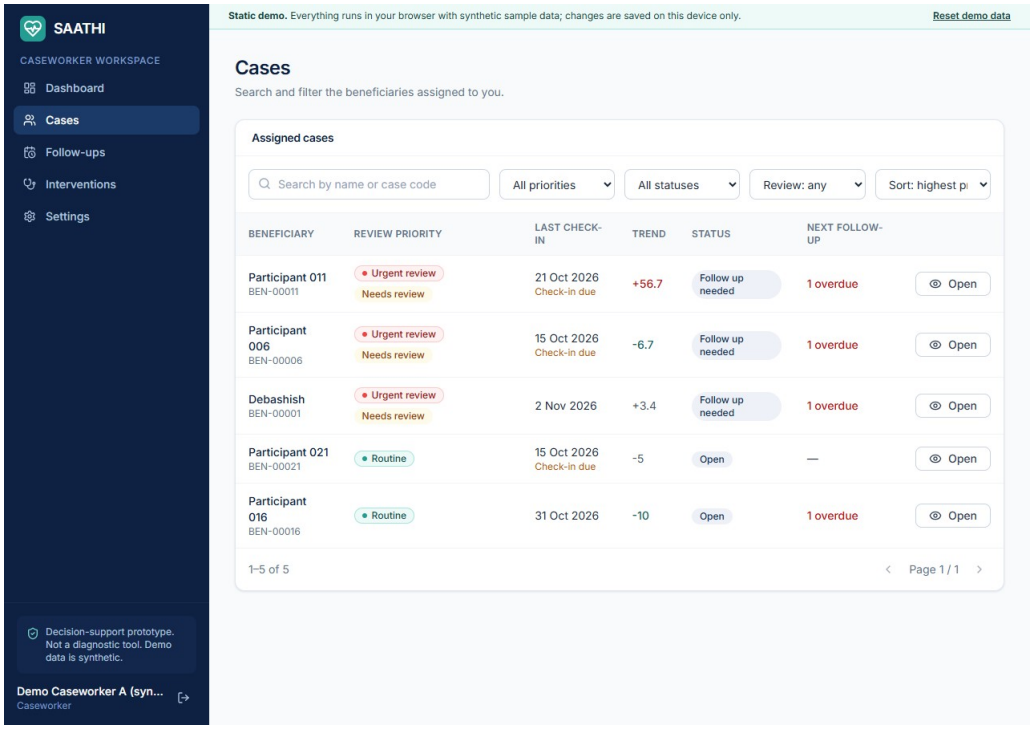


Figure 13: Assigned cases with search, filters, sort and pagination

Case profile

The case profile brings everything onto one screen. At the top are the review priority, status and any open urgent notification. The timeline chart plots the concern index with threshold lines and coloured dots for the level at each check-in. The radar chart compares the latest and previous dimension scores. **Why this review priority?** lists each factor that triggered the recommendation with its detail. The right-hand column has case notes, follow-ups (with Mark completed) and interventions (with an outcome selector), and the page offers Schedule follow-up, Record intervention and Mark latest check-in as reviewed.

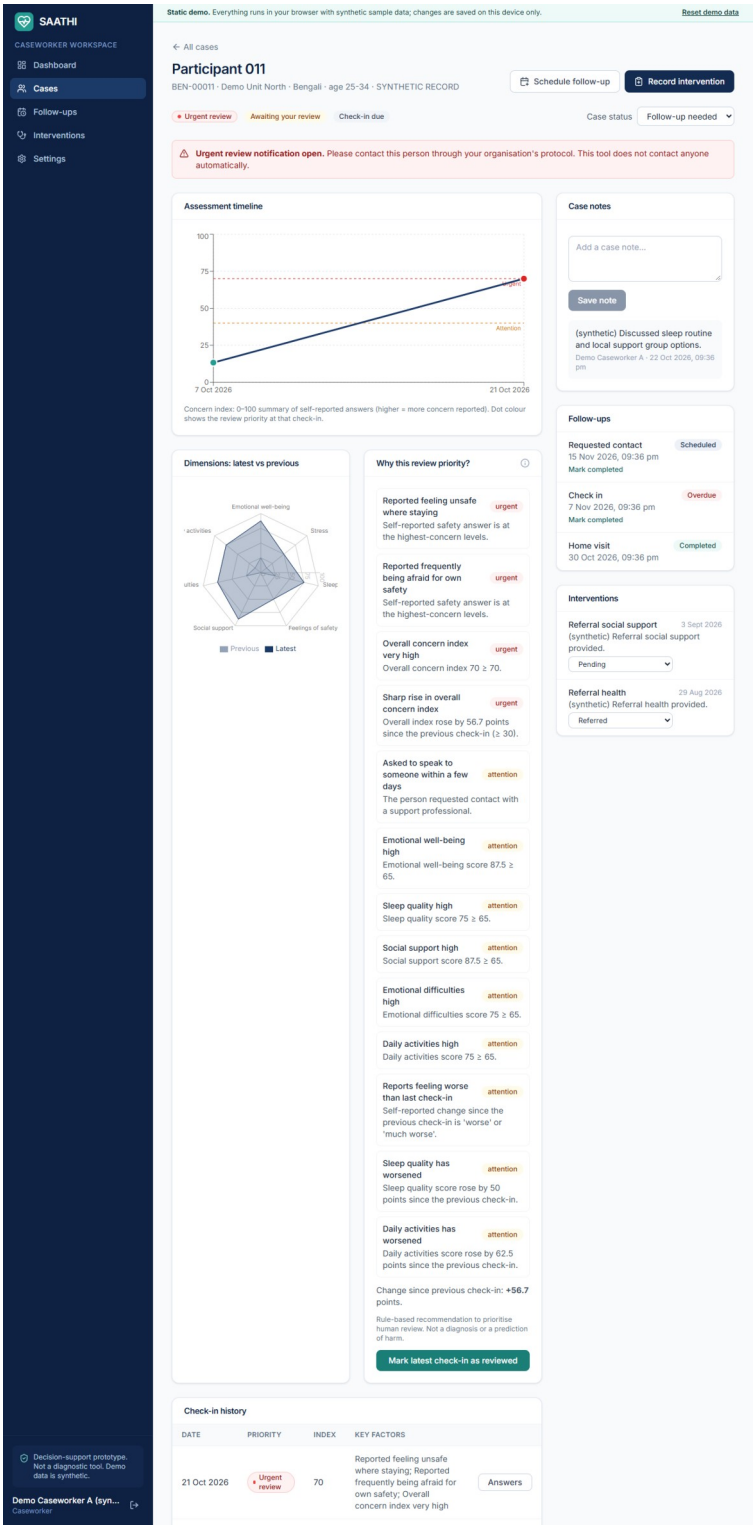


Figure 14: Case profile with timeline, dimensions and the explanation of the review priority

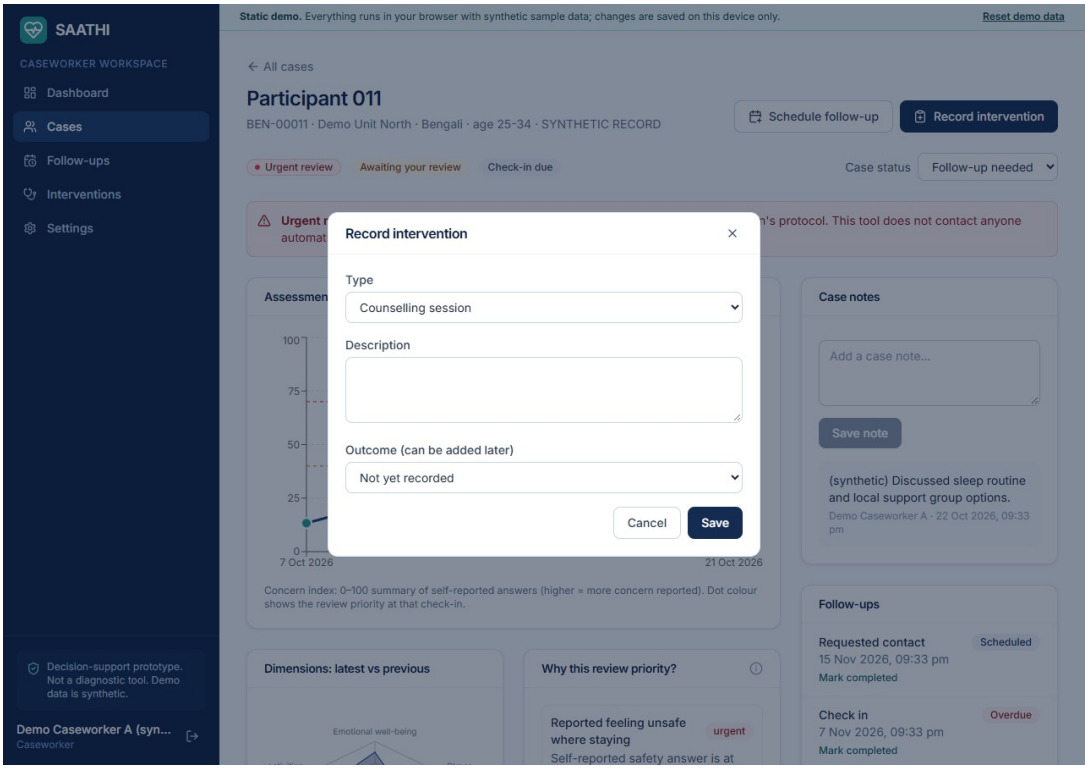


Figure 15: Recording an intervention

Follow-up management and intervention records

The follow-up page filters follow-ups into Requested (from beneficiaries), Due in 7 days, Overdue and Completed, and lets the caseworker schedule, reschedule or complete them. The intervention page lists all interventions for the caseload, filters by outcome and allows an outcome to be recorded later.

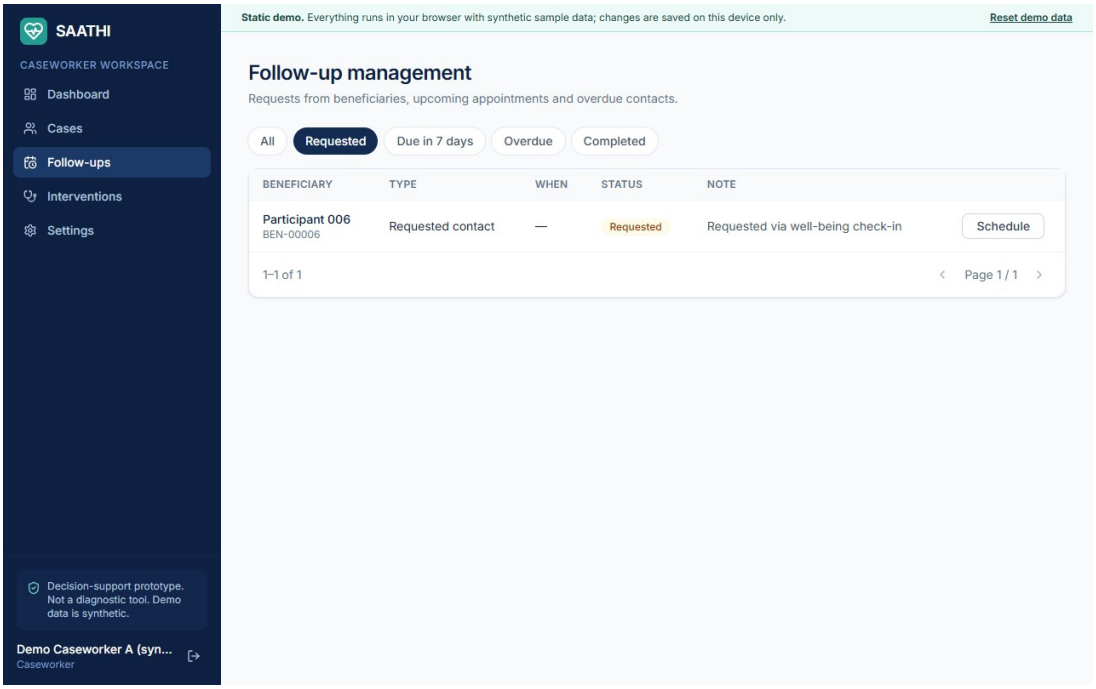


Figure 16: Follow-up management

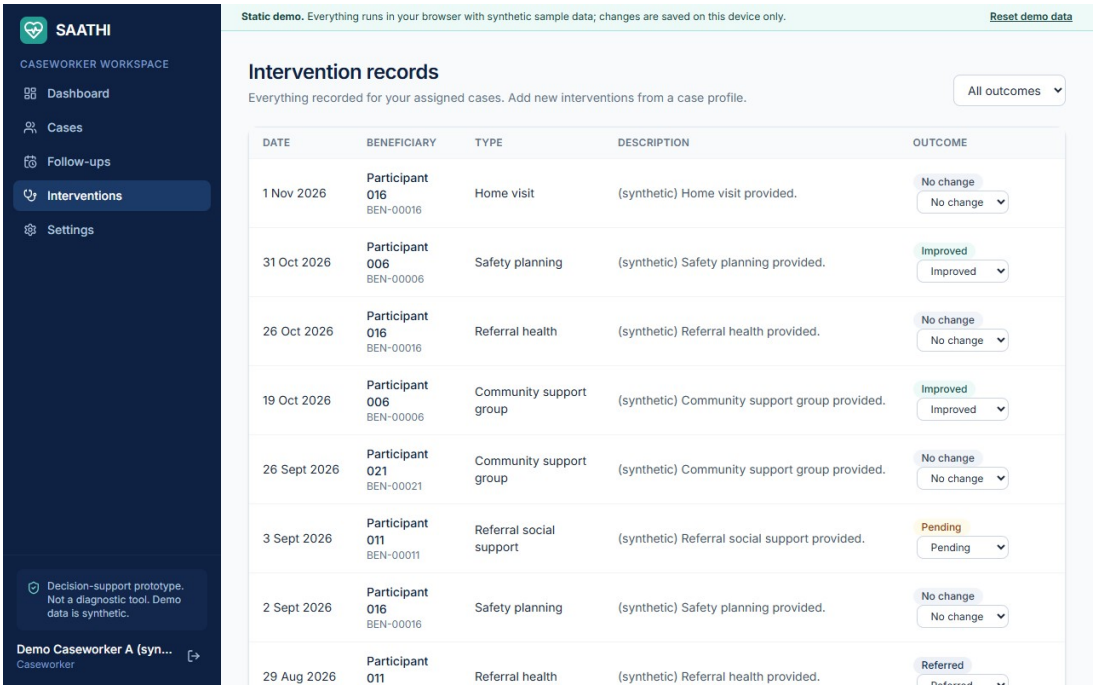


Figure 17: Intervention records

5.4 Administration

The administration area has five sections on one page. **Overview and assignments** shows headline numbers, caseloads and a drop-down to reassign a case. **Assessment engine** edits the thresholds with validation and shows the status of the optional machine-learning model. **Resources and emergency contacts** manages educational content, support services and emergency numbers with a Verified flag. **Onboarding and data deletion** creates single-use invitation codes, lists deletion requests and runs the retention job. **Audit log** lists sensitive actions with a filter.

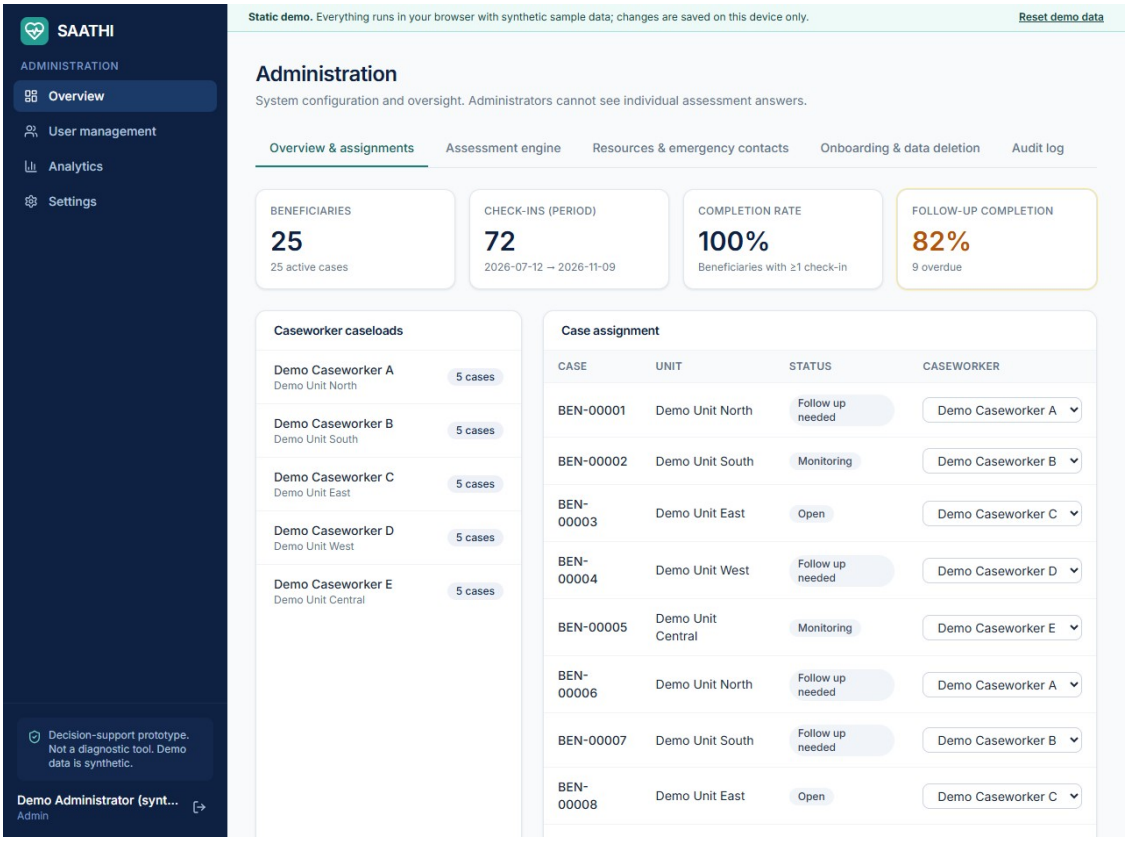


Figure 18: Administration overview and case assignment

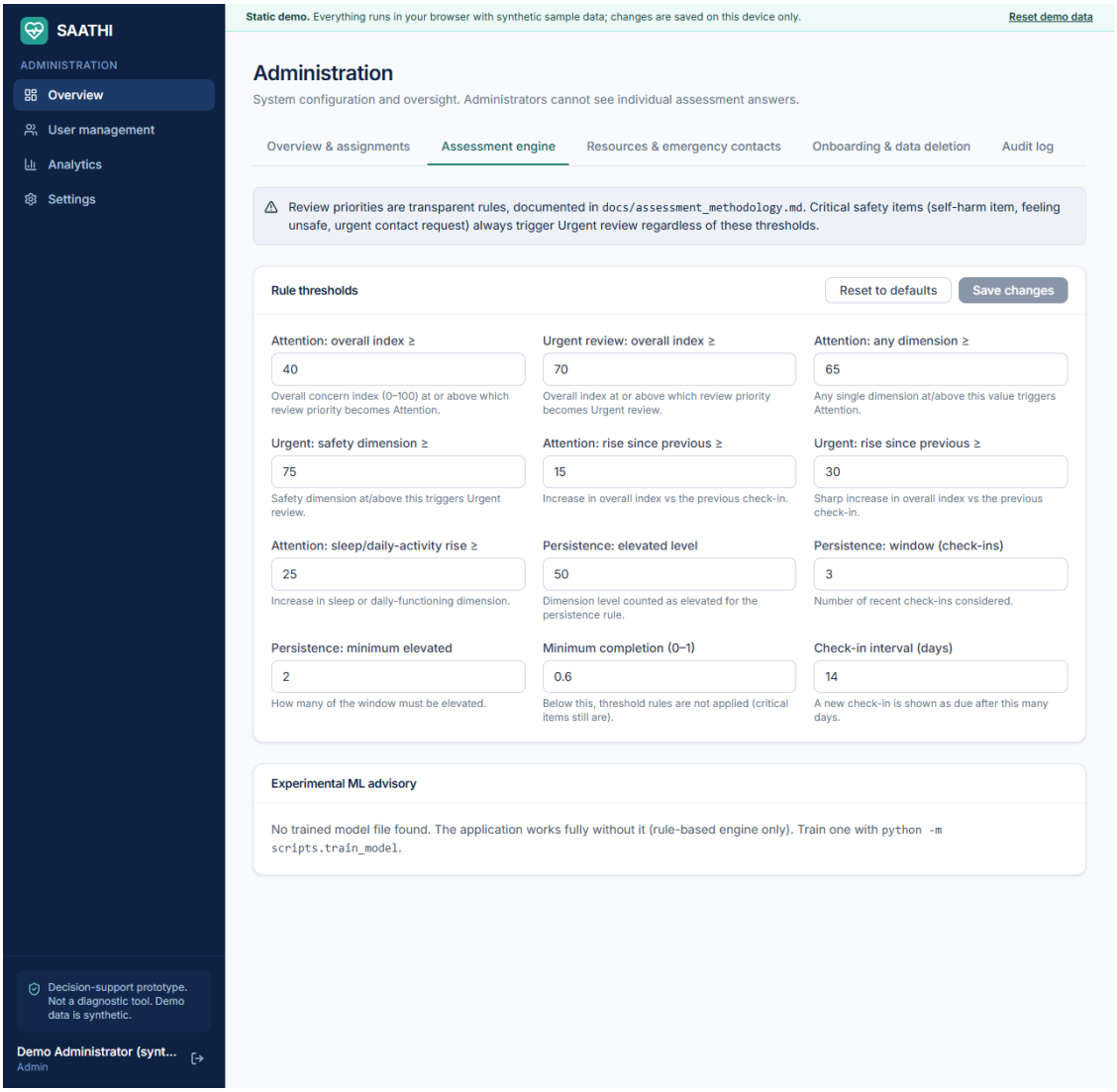


Figure 19: Engine configuration editor

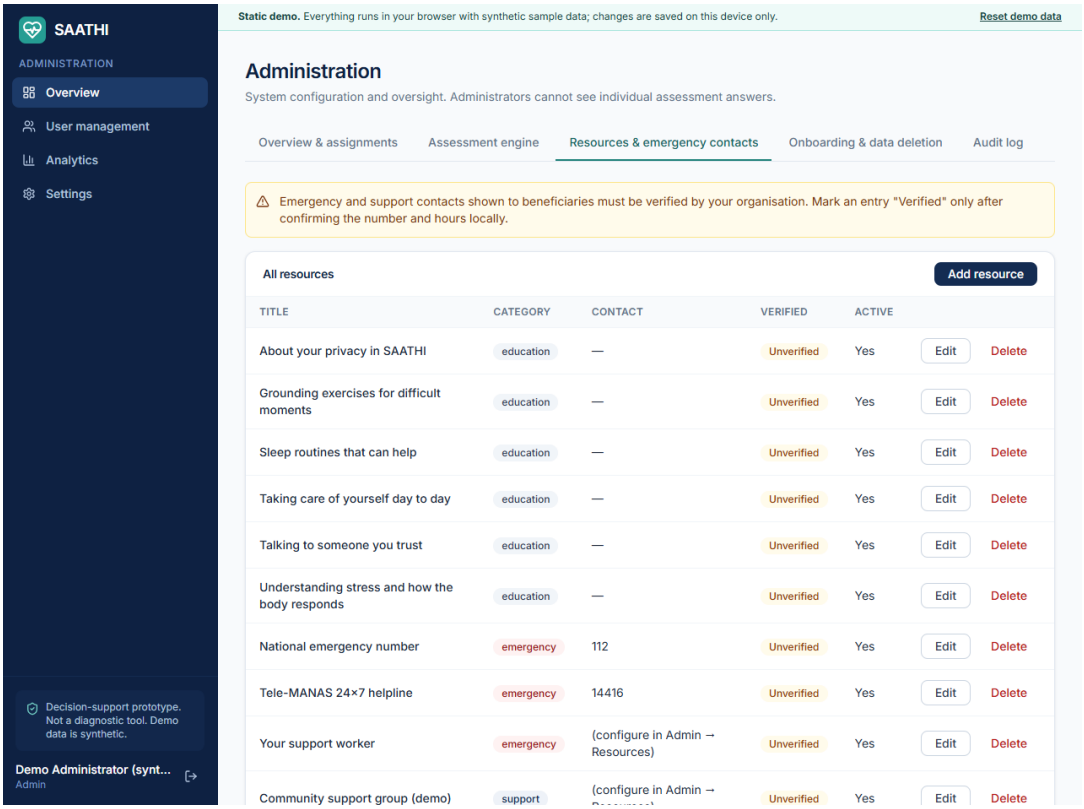


Figure 20: Resource and emergency-contact management

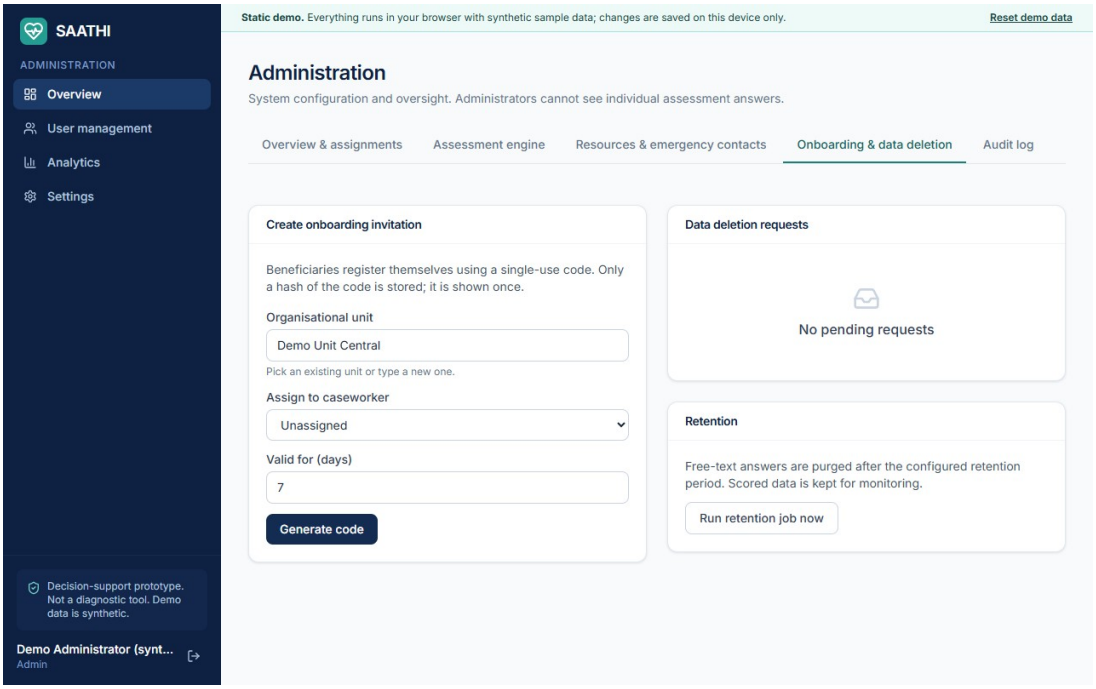


Figure 21: Onboarding invitations, deletion requests and retention

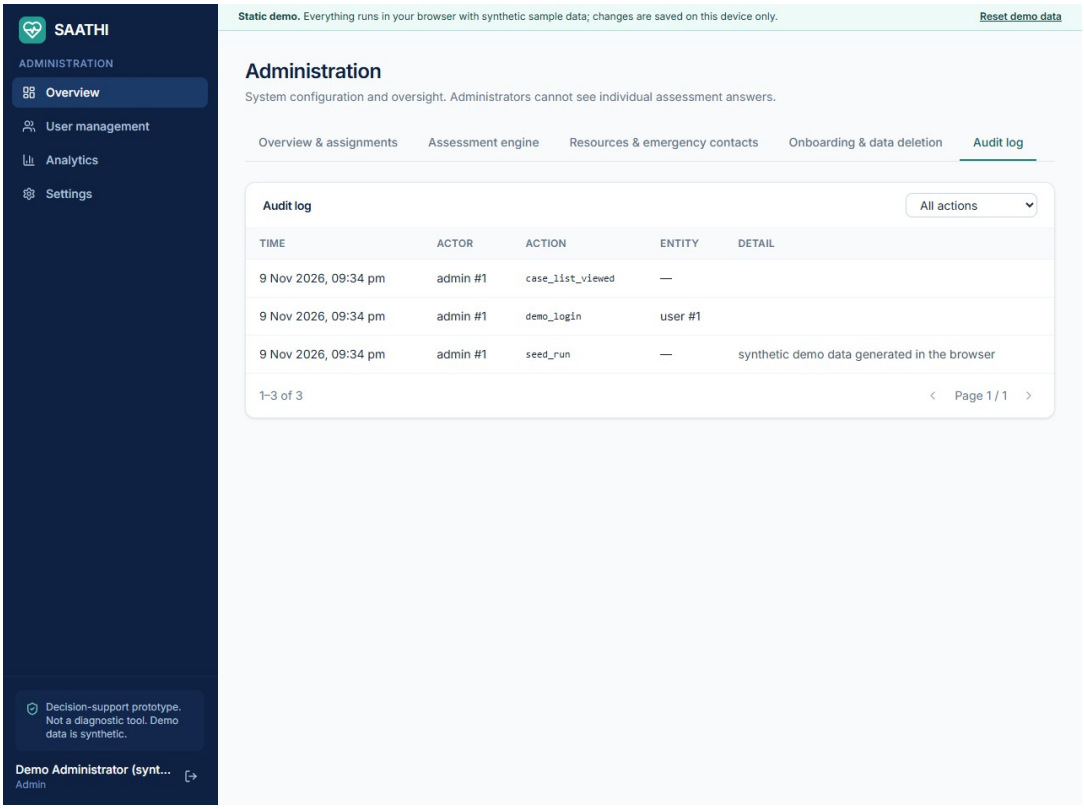


Figure 22: Audit log

In the static prototype the engine page reports that no machine-learning model is attached, because the machine-learning advisory is part of the Python backend (Section 7.5).

User management

Administrators can search and filter users, create staff accounts, change a role, deactivate or reactivate an account and reset a password. An administrator cannot deactivate or demote their own account. Beneficiaries cannot change role.

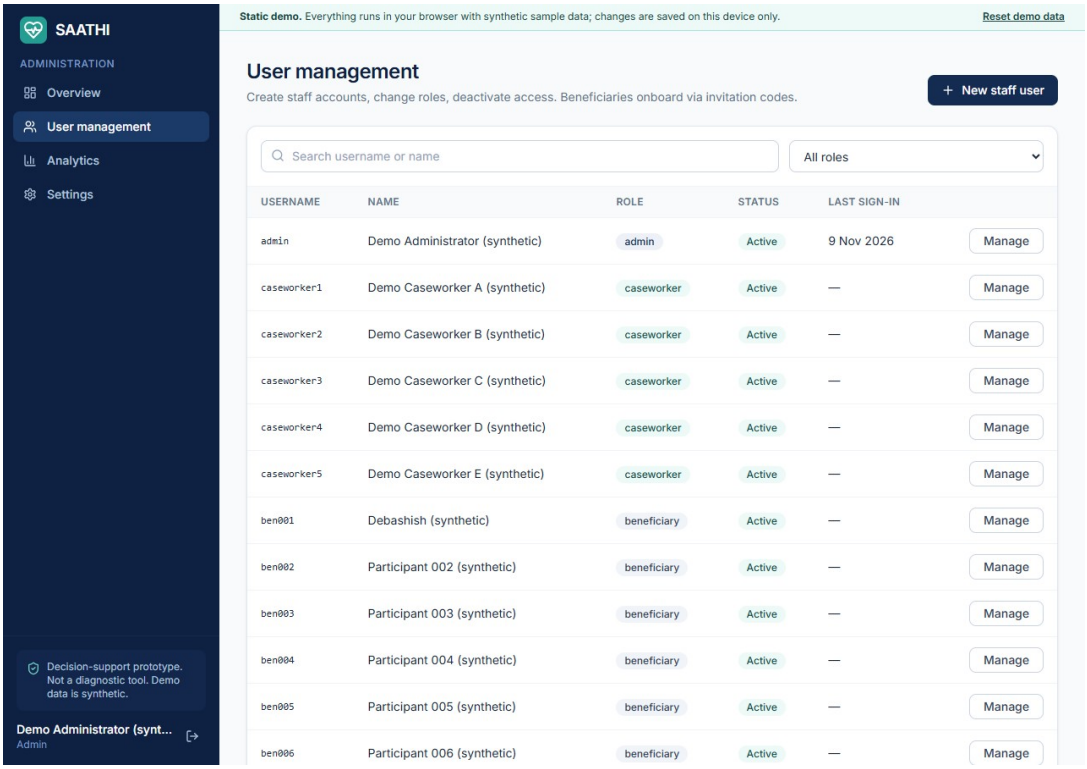


Figure 23: User management

Analytics

The analytics page offers date and organisational-unit filters and shows aggregated data only: completion rate, check-ins per week, the distribution of latest review levels among people who consented to analytics, the monthly average well-being trend, follow-up status and completion, and intervention types and outcomes. Groups smaller than three are hidden.



Figure 24: Aggregated analytics dashboard

5.5 Responsive design

The layout adapts to phones and tablets: below the desktop width the sidebar becomes a menu button that opens a navigation drawer, cards stack, and wide tables scroll inside their own container instead of widening the page. Automated tests check every page for horizontal overflow at 375, 768 and 1280 pixels.

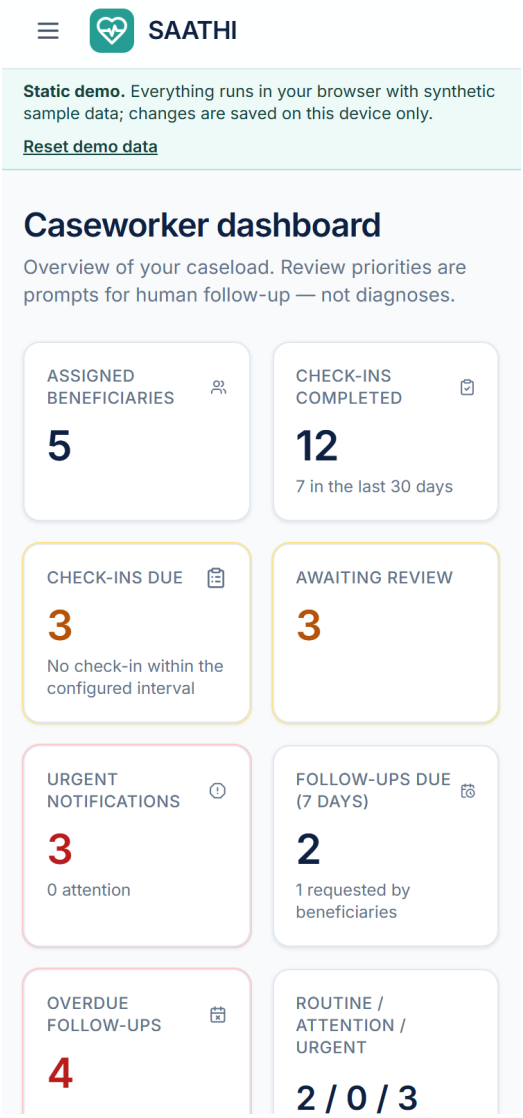


Figure 25: Caseworker dashboard on a 375-pixel-wide phone

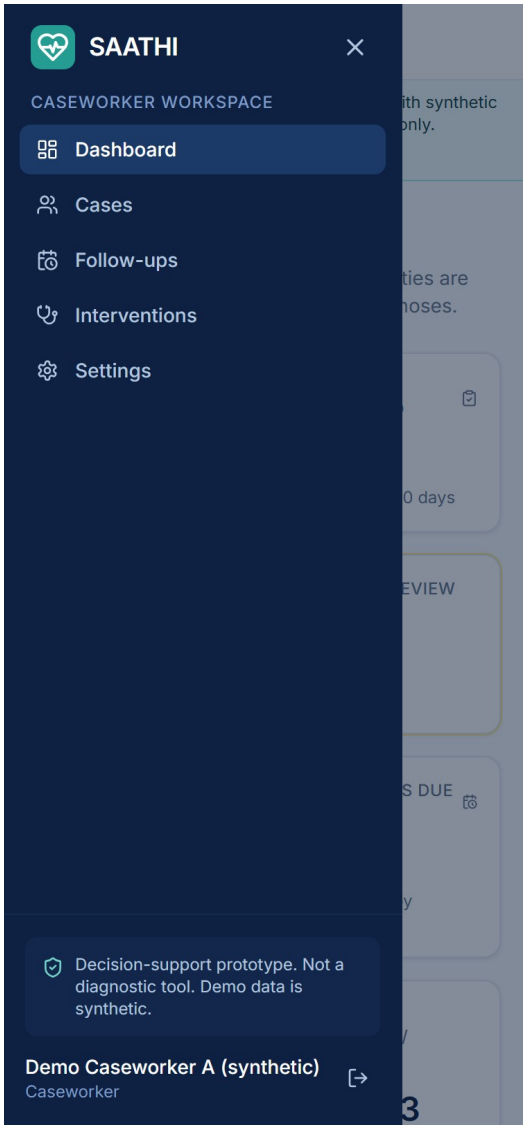


Figure 26: Navigation drawer on a phone

6. Assessment methodology

Read this first

The questionnaire is an original prototype (version `saathi-proto-1.0`). It has **not** been clinically validated. The output is a review priority that asks a person to look sooner or later. It is not a diagnosis and not a probability of harm.

6.1 The questionnaire

There are 18 items over a two-week look-back period: 17 required closed-ended questions (Likert scales and multiple choice) and one optional free-text box. Free text is stored for people to read and is **never scored or analysed** by the system. Fifteen of the closed-ended questions feed seven dimension scores; two (change since the last check-in, and the wish to speak to someone) are used directly by rules.

Dimension / group	Items	Example wording
Emotional well-being	2	How often have you felt low or down? How often have you felt calm and able to enjoy things? (reverse-keyed)
Stress	2	How often have you felt under a lot of stress? Overwhelmed by the things you need to handle?
Sleep quality	2	Trouble falling or staying asleep? How rested have you felt on waking? (reverse-keyed)
Feelings of safety	3	How safe have you felt where you are staying? (reverse-keyed) Afraid for your own safety? Thoughts of hurting yourself (critical item)
Social support	2	Someone to talk to when you needed support? (reverse-keyed) Felt isolated or alone?
Emotional difficulties	2	Upsetting memories or worries hard to put aside? Felt that things are too hard to keep going?
Daily activities	2	Difficulty with everyday activities? Able to get done what you planned? (reverse-keyed)
Change since last check-in	1	Compared with your last check-in, how have you been feeling overall?
Wish to speak to someone	1	Would you like to speak with a support professional?
Optional note	1	Anything else you would like your support worker to know? (free text, not scored)

Every answer maps to a **concern value from 0 (no concern) to 4 (most concern)** through an explicit table kept on the server, so reverse-keyed items are handled by mapping rather than by assumption. The mapping is not sent to the browser. For example, reverse-keyed frequency items map 0 to 4 and 4 to 0, the self-harm item maps its four options to 0, 2, 3 and 4, and the change item maps to 0, 0, 1, 3 and 4.

6.2 Scores

- **Dimension score (0 to 100)** = the mean concern of the answered items in that dimension divided by 4, times 100. Unanswered items are excluded; they are not counted as zero.

- **Overall concern index (0 to 100)** = the weighted mean of the available dimension scores. The safety dimension has weight 1.5 and the others 1.0; weights are re-normalised when a dimension is missing.
- **Well-being score** shown to beneficiaries = 100 minus the concern index, so that a higher number reads as better.

6.3 Review-priority rules

The level is the highest level triggered by any rule. The default thresholds below are configurable by an administrator, validated on save, and stored with every result so that each historical recommendation can be reproduced.

Level	Meaning	Triggers (any of)
Routine	No significant new indicators	None of the rules below fired.
Attention	Indicators warrant timely human review	Overall index at or above 40 Any dimension at or above 65 Wish to speak to someone within a few days Self-reported feeling a little worse or much worse than last time Overall index rose by 15 or more since the previous check-in Sleep or daily-activities dimension rose by 25 or more A dimension at or above 50 in at least 2 of the last 3 check-ins (persistence)
Urgent review	Needs prompt human assessment	Critical items: any non-zero answer to the self-harm item; feeling unsafe where staying, or afraid for own safety, at the two highest-concern answers; asking to be contacted as soon as possible Overall index at or above 70 Safety dimension at or above 75 Overall index rose by 30 or more since the previous check-in

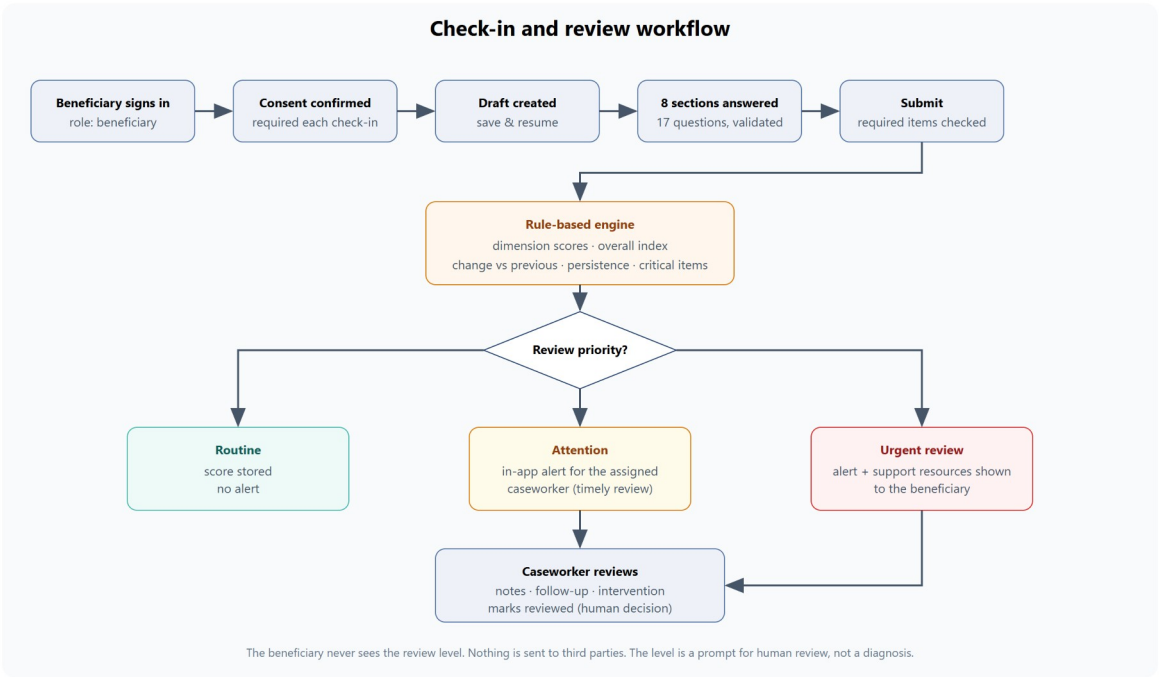


Figure 27: Check-in and review workflow

6.4 Missing data

If fewer than 60 per cent of the 17 closed-ended questions were answered, the threshold, change and persistence rules are **not applied**, because they would be unreliable, and a data-quality note is attached to the result. The critical-item rules are always evaluated on whatever was answered. An incomplete check-in therefore never silently turns into a plain Routine result, and it never hides a safety trigger.

6.5 Longitudinal monitoring

Earlier submitted and scored check-ins form the history for a new one. The engine reports the change in the overall index, the change in each dimension, and which dimensions have stayed elevated. Caseworkers can compare any two check-ins of the same person. If the new level is Attention or Urgent review, an in-app notification is created for the assigned caseworker. A request to speak to someone also creates a follow-up request. No message leaves the system.

6.6 What the engine does not use

The engine's function signature accepts only the answers, the person's own history and a configuration. It has no parameter for age band, language, organisational unit, caste, religion, gender or victim status, and a unit test asserts this.

7. Backend design (full system)

> **Scope note**
> This section documents the FastAPI backend, database and machine-learning component that make up the complete system in the repository. They are **not used by the static prototype link**, which serves the same screens from an in-browser implementation (Section 8). They were run and tested separately, as reported in Section 10.

7.1 REST API

The API is organised by domain and documented automatically by Swagger UI (`/docs`) and ReDoc (`/redoc`). All paths are prefixed with `/api`. Requests and responses are validated with Pydantic schemas. Errors follow one convention: 401 not authenticated or token revoked, 403 role not allowed, 404 not found or not your case, 409 state conflict, 422 validation (a structured list of field errors), 429 temporary lock-out, and 500 with a generic message (details only in the server log). The OpenAPI schema reports **64 operations**.

Group	Operations	Purpose
Authentication	7	Login, logout, current user, change password, invitation-based registration, public configuration, demo-mode sign-in
Beneficiaries	5	Create, list (basic roster), own dashboard data, read, update permitted fields
Cases	5	Filtered and paginated case list, single-screen case profile, status change, notes, assignment
Assessments	11	Questionnaire, draft and resume, save answers, submit, history, detail, explanation, compare, recalculate, mark reviewed
Follow-ups	4	Request or schedule, list with filters, update, complete
Interventions	4	Types, record, list, record outcome
Alerts	2	List review notifications, acknowledge or resolve
Analytics	3	Caseworker summary, administrator overview, trends
Administration	17	Users, caseworkers, organisational units, invitations, resources (CRUD), engine configuration, audit log and actions, deletion requests, erasure, retention
Self-service	5	Resources, consent state and changes, deletion request, data export
System	1	Health check (database connectivity, ML model availability)

A full list of operations is in Appendix A.

7.2 Authentication and role-based access

- **Passwords** are hashed with bcrypt (12 rounds) and checked against a strength rule. They are never logged or returned.
- **Tokens** are JWTs (HS256) valid for 60 minutes. Each user has a token version: logout, password change, deactivation and role change increase it, so all earlier tokens stop working immediately.
- **Brute-force protection:** five failed logins lock the account for 15 minutes. A dummy hash check keeps the timing the same when a username does not exist.

- **Role checks** run on every route through dependency functions. A second, **object-level check** decides whether the caller may touch a particular beneficiary: a beneficiary only their own record, a caseworker only an actively assigned case, an administrator only non-clinical profile data. A caseworker asking for someone else's case receives 404 rather than 403, so that the existence of the case is not revealed.
- **Registration** needs a single-use, expiring invitation code created by staff. Only a SHA-256 hash of the code is stored.

7.3 Database design

The schema has **16 tables**, defined as SQLAlchemy models and created by one Alembic migration. Primary and foreign keys, unique constraints, CHECK constraints on enumerated values, and indexes on the common query paths enforce integrity. Timestamps use a custom type that stores UTC and always returns timezone-aware values on both SQLite and PostgreSQL. Answers are stored as rows (not a single blob) so that individual items can be analysed or purged; derived scores and explanations are stored per assessment with the engine version and configuration snapshot.

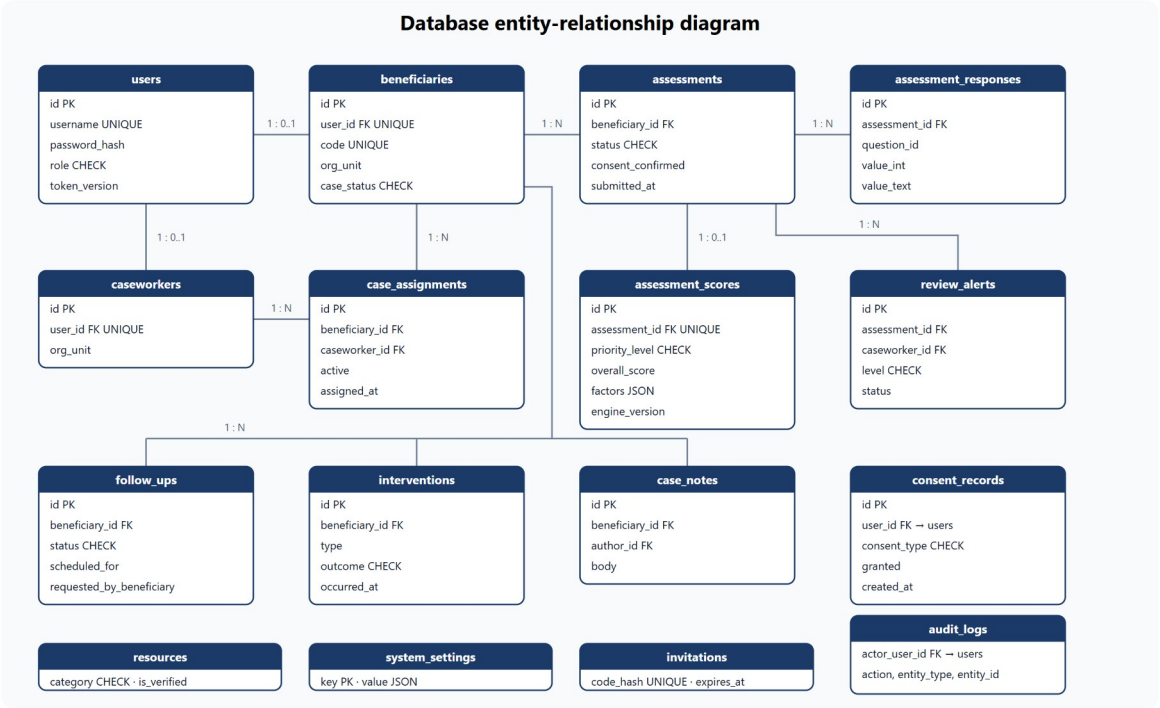


Figure 28: Entity-relationship diagram of the main tables

Table	Purpose
users	Accounts: username, display name, bcrypt hash, role, active flag, token version, lock-out fields
beneficiaries	Pseudonymous case code, organisational unit, coarse age band, language, case status, deletion-request time
caseworkers, case_assignments	Caseworker profile; history of which caseworker is assigned to which beneficiary (one active at a time)
assessments, assessment_responses	Check-in header (status, consent, timestamps) and one row per answer

Table	Purpose
assessment_scores	Engine output: review level, concern index, dimension scores, factors (the explanation), change summary, data quality, engine version, configuration snapshot, optional ML advisory, review status
review_alerts	In-app notifications for the assigned caseworker (attention or urgent) with status
follow_ups, interventions, case_notes	Case-management records
resources	Education, support and emergency entries with a Verified flag
consent_records	Append-only history of consent choices
audit_logs	Who did what to which record; never contains answers or secrets
invitations, system_settings	Hashed onboarding codes; administrator-managed configuration such as the engine thresholds

Retention and deletion. An administrator can erase a beneficiary and all linked records (assessments, notes, follow-ups, interventions, alerts, consents) in one operation. A retention job clears free-text answers older than a configured period (365 days by default) while keeping scored data for monitoring. Beneficiaries can request deletion, which flags the record and withdraws analytics consent at once.

7.4 The assessment engine module

The engine lives in its own Python package (`backend/app/engine`) and consists of **pure functions**: they take answers, history and a configuration and return a result, with no database or network access. That keeps them easy to test and lets the engine be replaced (for example by a clinically validated instrument) without rewriting the application. A thin service layer loads earlier assessments, calls the engine, stores the result, and creates notifications and follow-up requests.

7.5 Machine-learning advisory (experimental)

> **Honest limits**
> The model is trained on **synthetic data generated by this project's own code**. It learns the generator's assumptions, not real distress. The reported scores are **not evidence of clinical or real-world predictive validity** and the model must not be used for decisions. It never changes the review level.

- **Synthetic dataset.** 6,000 samples with a fixed seed (42). Each sample draws a latent support-need value from a Beta(1.6, 2.2) distribution and generates the answers from it with noise; 25 per cent of samples have about 15 per cent of items missing; 60 per cent have a simulated previous check-in so that change features exist. Features are computed with the same engine code used in production: seven dimension scores, overall index, change, a has-previous flag, help-request concern and the self-harm flag. No demographic features are used.
- **Labels** come from the latent state plus noise (cut-points 0.35 and 0.62), not from the rule engine's output, so the model is not simply re-learning the rules.

- **Model.** Standardisation followed by multinomial logistic regression, chosen because it is interpretable. A 75/25 stratified split is used and a majority-class baseline is reported.
- **Explainability.** For the predicted class, the top four features by coefficient times scaled value are shown as influences.
- **Optional by design.** If the model file is missing, the application works unchanged (covered by a unit test).

Metric (synthetic held-out set, 1,500 samples)	Value
Accuracy	0.7913
Macro F1	0.7855
ROC-AUC (one-vs-rest, macro)	0.9305
Majority-class baseline accuracy	0.416
Agreement of model class with the rule engine's level	0.7047

Retraining with the same seed reproduced the metrics file byte for byte.

7.6 Analytics and privacy controls

- Administrator analytics return **aggregates only** and never identifiers.
- Counts below a minimum cell size (default 3) are suppressed and returned as empty values, so small groups cannot be identified.
- Well-being and priority views include only people whose latest analytics consent is granted.
- Filters by period and organisational unit are applied on the server.

7.7 Administration, configuration and audit

Administrators manage users (create, change role, deactivate, reset password), organisational units, onboarding invitations, educational resources and emergency contacts (with a Verified flag), and the engine thresholds. Saving a configuration is rejected with specific messages if, for example, the attention threshold is not below the urgent threshold. The audit log records logins (including failures), staff views of cases and assessments, status changes, assignments, configuration changes, consent changes, exports and erasures. Audit details never contain answers or passwords.

7.8 Seed data

A one-command script (`python -m scripts.seed --reset`) drops the tables, re-runs the migration and creates the synthetic demonstration data: 1 administrator, 5 caseworkers, 25 beneficiaries, 84 historical assessments across several trajectory types (stable low, stable mid, stable high, improving, declining, volatile), 53 follow-ups (completed, upcoming, overdue and requested), 49 interventions with outcomes, case notes and 12 resources. Every record is flagged as synthetic. Demo passwords come from environment variables and are never hard-coded.

7.9 Operations

- Configuration comes from environment variables (`.env`, never committed). In production mode the application refuses to start with a weak signing key.

- Security headers (no-sniff, frame denial, no referrer, no caching) are added to every response; CORS allows only configured origins.
- A generic error handler hides internal details; request bodies are never logged.
- A health endpoint reports database connectivity and whether the ML model is loaded. A bootstrap script migrates, seeds an empty database once, and trains the optional model, which is how the hosted configuration starts.
- Docker and nginx files are provided; see Section 10 for their verification status.

8. The static demo build used for the prototype link

A full-stack deployment needs a Python server and a database. To let anyone open the prototype from a simple link, the same React application can be built in **static demo mode** (`VITE_STATIC_DEMO=true`). In this mode the application's HTTP client does not use the network. A custom Axios adapter hands each `/api` request to an in-browser implementation of the API.

8.1 What runs in the browser

- A **TypeScript port of the assessment engine**: the same questionnaire, concern mappings, dimension scores, overall index, rules, thresholds, missing-data handling and explanations as the Python engine.
- A **router and handlers** for every endpoint the interface uses, with the same response shapes and the same role and object-level checks (for example a caseworker opening someone else's case gets 404, and an administrator is refused case content with 403).
- A **seeded synthetic database** with 25 beneficiaries, 5 caseworkers, an administrator, more than 50 check-ins on different trajectories, follow-ups, interventions, notes and resources, generated with a fixed seed.
- **Persistence in the browser** (`localStorage`) so that changes survive a page reload; a Reset demo data button in the banner restores the original sample data.
- Sign-in by demo buttons, or by a sample username with the sample password.

8.2 Differences from the full system

Aspect	Full system (repository backend)	Static prototype
Where logic runs	FastAPI server	In the visitor's browser
Data storage	SQLite or PostgreSQL, shared by all users	localStorage, private to each visitor and device
Passwords and tokens	bcrypt hashes, signed JWTs	Sample password for synthetic accounts; simple session tokens. Not for real use
ML advisory	Available when a model has been trained	Not included
Audit log and analytics	Stored server-side	Implemented against the browser data
Network calls to /api	Yes	None (verified by a test)
Suitable for real beneficiary data	Only after the security and legal steps in Section 9	No

Why this is stated plainly

The static build is a faithful **demonstration** of the interface and the rules, not a secure deployment. Because the data and the "server" live in the visitor's browser, nothing a visitor does affects anyone else, and nothing is protected by server-side security. This is acceptable for synthetic data and unsuitable for real data.

9. Security, privacy and ethics

SAATHI handles highly sensitive information by design, but it is a prototype: it has had no external security review, penetration test or legal data-protection assessment. The controls below are implemented in the full backend and tested as described in Section 10.

Concern	Control	Where
Passwords	bcrypt, strength check, never logged	security module
Authentication	Signed JWTs with expiry and server-side revocation through a token version	security, deps
Brute force	5 failed attempts lock the account for 15 minutes; constant-time style check for unknown users	auth router
Authorisation	Role checks on every route plus object-level checks; 404 for cases that are not yours; administrators cannot read answers	deps
Injection	Parameterised queries through SQLAlchemy; validated, typed request bodies	everywhere
Cross-site scripting	React escapes output; no raw HTML rendering; no third-party scripts	frontend
Headers and CORS	nosniff, frame denial, no referrer, no caching; explicit CORS origins	main, nginx
Secrets	Environment variables only; production refuses a weak signing key; no credentials in the repository	config
Error leakage	Generic 500 responses; stack traces only in the server log; request bodies never logged	main
Consent	Required at registration and at the start of every check-in; append-only consent history	auth, assessments, me
Audit	Sensitive access and changes are logged without content	audit service
Data rights	Export, deletion request, administrator erasure, retention of free text	me, admin
Analytics privacy	Aggregate only, consent-aware, small-cell suppression	analytics

9.1 Ethical safeguards

- The beneficiary never sees the review level; they see a well-being score and resources.
- Critical answers always lead to visible support information and a notification to a human, and the screen states that the application is not an emergency service.
- Emergency and support contacts are managed by administrators and show whether they have been verified. The demo entries are unverified placeholders.
- The engine excludes demographic attributes; the ML component is clearly separated, labelled experimental and advisory only.
- Only synthetic data is used; no real beneficiary records exist anywhere in the repository.

9.2 Residual risks

- Tokens are kept in `sessionStorage`, which scripts injected into the page could read. A production system should prefer HTTP-only cookies with CSRF protection and a strict content-security policy.
- There is no multi-factor authentication, e-mail password reset, token refresh or IP rate limiting; the account lock-out could itself be used to lock out a known user.
- Data at rest is not encrypted at the application level; audit logs are not tamper-evident; free text may contain identifying details typed by users.
- HTTPS must be provided by a reverse proxy or the hosting platform.
- The privacy notice is prototype text. A legally reviewed notice (for example under India's Digital Personal Data Protection Act, 2023) and a data-protection impact assessment are needed before any real deployment.

10. Testing and verification

All results below were obtained by running the tests on 9 November 2026 on Windows 11 with Python 3.12 and Node.js 26. Where something was not run, it is said so.

Suite	Result	What it covers
Backend unit and API tests (pytest)	51 passed	Engine rules (routine, attention, urgent, critical item with sparse data, missing data, rise, persistence, configuration, no demographic inputs, optional ML); authentication (login, lock-out, logout revokes tokens, password change, invitation registration); role and object-level access; check-in workflow; follow-ups, interventions, notes; case-list filters and paging; analytics privacy; user management; assignment; configuration validation; resources; audit log; consent, export, erasure; retention; demo mode.
Live API audit script	256 checks passed; all 64 operations exercised	A black-box script drives a running server over HTTP, tests every operation with a role matrix and bad inputs, reads the database directly after writes, then (after restarting the server) confirms persistence and erasure cascades and runs integrity checks. It reports any operation it did not exercise: none.
Same audit on PostgreSQL 16.2	Passed (247 checks, 60 operations at that time)	Run before the organisational-unit, audit-action and demo-mode endpoints were added. Migrations, seed, the pytest suite (48 tests at that time) and the live audit all passed. No server errors appeared in the logs. No schema drift was reported by Alembic.
Frontend unit tests (Vitest)	20 passed	UI components, error mapping, login behaviour and consent gating (9 tests); the in-browser API and engine port: parity of the rules, role and access rules, check-in workflow, persistence after reload, analytics privacy, administration (11 tests).
Type check and production build	No errors	TypeScript strict mode; Vite build for both the normal and the static-demo configuration.
End-to-end tests against the full stack (Playwright, Edge)	38 tests	Every user workflow through the real UI and API, navigation and route guards for each role, responsive layouts at 375, 768 and 1280 pixels, loading, network-failure, 500 and 403 states, and a check that the numbers on screen match the API. The last complete run of 37 tests passed 35; the two failures were timing races in the test code, which were corrected, and each corrected test was then re-run and passed. The demo-mode test was added afterwards and passed.
End-to-end tests of the static build	6 passed	Runs with no backend at all: one-click roles, typed login, a critical-answer check-in followed by the caseworker notification, persistence after reload, administration actions, reset, and phone layout. Also asserts that no request is sent to `/api`.
Fresh-install check	Passed	A clean copy of the repository (earlier version) was installed from the documented steps, seeded, tested and built.

10.1 Defects found and fixed during verification

#	Defect and fix
1	Comparing a check-in with a check-in of a different beneficiary returned an empty 200 response. It now returns 422, and a check-in from a case the caller does not hold still returns 404.
2	On phones, caseworker and administrator pages could be scrolled sideways: visually hidden table headers inside scrolling tables widened the page. Fixed in the style sheet; verified at 375 pixels.

#	Defect and fix
3	The caseworker dashboard silently showed nothing when its summary request failed. It now shows the error with a retry button.
4	The default organisational unit and the audit-action filter were hard-coded in the interface. They now come from the API.
5	The documentation described the questionnaire as 17 items; it has 18 (17 closed-ended and one free-text box). Documentation corrected.

10.2 Not run or not covered

- **Docker:** Docker Desktop was installed, but its engine could not start until WSL2 is enabled and the computer is restarted. The Dockerfiles and Compose file were reviewed by reading only; they have not been built or run.
- Automated accessibility audits, load and performance tests, and penetration testing.
- Browsers other than Microsoft Edge; macOS and Linux (commands were executed on Windows only).
- The hosted deployments on Render and Netlify; configuration files are provided and the pieces were tested locally.

11. Repository, deployment and reproducibility

11.1 Source code

GitHub repository: https://github.com/drave10/SAATHI

The repository contains the backend, frontend, documentation and deployment configuration. It excludes secrets, databases, virtual environments and build output through `.gitignore`. The `docs/` folder holds the architecture, database, API, methodology, machine-learning, security, testing, reproducibility and future-scope documents, written to match the implementation.

11.2 Ways to run the project

Mode	How
Full stack, local	Copy `.env.example` to `.env`; create a Python virtual environment; `pip install -r requirements.txt`; `python -m scripts.seed --reset`; `uvicorn app.main:app --port 8000`; in `frontend/` run `npm ci` and `npm run dev`. Swagger UI is at `/docs`.
Static prototype, local	In `frontend/` run `npm ci`, then build with `VITE_STATIC_DEMO=true` and preview, or run the dev server with that variable set. No backend is needed.
Static prototype, hosted	`frontend/netlify.toml` configures Netlify: build command, publish folder, the single-page-app fallback and `VITE_STATIC_DEMO=true`. Connect the GitHub repository in Netlify with base directory `frontend`, or build locally and upload the `dist` folder.
Full stack, hosted	`render.yaml` describes a Render web service and PostgreSQL database; a bootstrap script migrates, seeds an empty database once and trains the optional model. The frontend is then built without the static flag and /api is proxied or rewritten to the API.
Docker	`docker compose up --build` (not yet executed, see Section 10.2).

11.3 Reproducibility

- Python and JavaScript dependencies are pinned to exact versions; the frontend has a lock file.
- The database schema comes from a migration; the demonstration data comes from fixed random seeds (2026 for the sample data, 42 for the ML dataset). Dates in the sample data are relative to the day it is generated.
- One command resets and regenerates the demonstration environment.
- Tests and the live audit scripts are included so that a reviewer can repeat the verification.

12. Known limitations

- The questionnaire is an original prototype, not a clinically validated instrument; thresholds are illustrative and have not been calibrated against clinical outcomes. No sensitivity or specificity is claimed.
- The machine-learning component is trained on synthetic data produced by the project's own generator. Its metrics say nothing about real-world validity.
- Free-text answers are stored and shown to caseworkers but are not analysed automatically.
- The emergency and support contacts in the sample data are unverified placeholders. An administrator must verify them before any real use.
- Notifications are in-app only. There is no SMS or e-mail, by design.
- The static prototype keeps data in the visitor's browser, has no server-side security and does not include the ML advisory. It must not be used with real data.
- The questionnaire is available in English only; no cultural adaptation or translation has been done.
- Case lists are computed per request in application code, which is fine for hundreds of cases but needs database-side aggregation at larger scale.
- Docker was not run, and no external security, accessibility or legal review has been performed.

13. Future scope

Clinical and methodological

- Replace or complement the prototype questionnaire with a licensed, validated instrument after verifying its terms, co-designed with clinicians and survivors' representatives, with translation and cultural adaptation.
- Prospective validation and calibration of the thresholds with caseworkers; fairness audits once real, ethically approved data exists.
- Human-reviewed assistance for reading free text, using validated and locally hosted models only.

Product

- Multilingual and low-literacy modes; an offline-first mobile app for field workers.
- A scheduling calendar and reminders, and (with explicit consent) SMS or WhatsApp reminders.
- A supervisor role, hand-over workflows and referral tracking with partner organisations.

Engineering and security

- HTTP-only cookie sessions with CSRF protection, multi-factor authentication, password reset, rate limiting and a strict content-security policy.
- Field-level encryption, key management, tamper-evident audit logs and scheduled retention jobs.
- Database-side aggregation and managed PostgreSQL; structured logging and metrics; continuous integration with dependency scanning and automated accessibility checks.
- A data-protection impact assessment and a legally reviewed privacy notice before any real deployment.

14. Conclusion

SAATHI shows how a support organisation could notice early changes in self-reported well-being and put scarce human attention where it is needed first, while keeping decisions with people. The prototype combines a gentle check-in experience for beneficiaries, a caseworker workspace that explains why a recommendation was made, and privacy-conscious administration and analytics.

The complete system in the repository includes a role-protected REST API with 64 operations, a 16-table database with migrations, a transparent and configurable assessment engine, an optional and clearly labelled machine-learning advisory, and an automated test and audit toolkit. The prototype interface can be demonstrated through a self-contained static build that needs no server.

The project is deliberately honest about its limits: the questionnaire is not clinically validated, the machine-learning model is trained on synthetic data, the sample contacts are placeholders and the demo holds only synthetic data. Those limits define the work needed before any real-world pilot: clinical co-design and validation, a security and legal review, and verified local support contacts.

Appendix A: API reference

All paths are under `/api`. Roles: B beneficiary, C caseworker, A administrator. The static prototype implements every operation that the interface calls.

Operation	Allowed roles	Purpose
POST /auth/register	public + invitation	Register with a single-use code and consent
POST /auth/login	public	Login; returns a JWT and the user
POST /auth/logout	any	Revoke all tokens of the user
GET /auth/me	any	Current user
POST /auth/change-password	any	Change password; revokes old tokens
GET /auth/config	public	Whether demo mode is on
POST /auth/demo-login	public (demo mode)	Passwordless sign-in for synthetic accounts only
POST /beneficiaries	C, A	Create a beneficiary
GET /beneficiaries	C, A	Basic roster without clinical data
GET /beneficiaries/me	B	Own dashboard data
GET /beneficiaries/{id}	B own, C assigned, A profile	Profile
PATCH /beneficiaries/{id}	B limited, C, A	Update permitted fields
GET /cases	C, A (roster only)	Search, filter, sort, paginate
GET /cases/{id}	C assigned	Whole case profile in one response
PATCH /cases/{id}/status	C	Change case status
POST /cases/{id}/notes	C	Add a case note
POST /cases/{id}/assign	A	Assign or reassign a caseworker
GET /assessments/questionnaire	any	Questionnaire without the scoring map
POST /assessments	B	Create or resume a draft (consent required)
GET /assessments/draft	B	Current draft with answers
PUT /assessments/{id}/responses	B	Save partial answers with validation
POST /assessments/{id}/submit	B	Validate, score, notify; returns well-being score and any support resources
GET /assessments	B own, C assigned	History (beneficiaries never receive the review level)
GET /assessments/{id}	B own, C assigned	Detail including answers
GET /assessments/{id}/explanation	C	Factors, change, data quality, ML advisory, disclaimer
GET /assessments/{id}/compare	C	Deltas against the previous or a chosen check-in
POST /assessments/{id}/recalculate	C	Re-run the engine with the current configuration
POST /assessments/{id}/mark-reviewed	C	Record human review
POST /followups	B request, C schedule	Create a follow-up
GET /followups	B own, C assigned	List with status, due, overdue filters

Operation	Allowed roles	Purpose
PATCH /followups/{id}	B cancel own, C	Reschedule or update
POST /followups/{id}/complete	C	Mark completed with notes
GET /interventions/types	C	Allowed intervention types
POST /interventions	C	Record an intervention
GET /interventions	C	Interventions for assigned cases
PATCH /interventions/{id}/outcome	C	Record an outcome
GET /alerts	C	Review notifications (urgent first)
PATCH /alerts/{id}	C	Acknowledge or resolve
GET /analytics/caseworker-summary	C	Caseload metrics and recent activity
GET /analytics/overview	A	Aggregated, anonymised statistics
GET /analytics/trends	A	Chart subset of the overview
GET, POST /admin/users; PATCH /admin/users/{id}	A	List, create staff, update role / status / password
GET /admin/caseworkers	A, C	Caseworkers with caseload
GET /admin/org-units	A, C	Organisational units in use
POST /admin/invitations	A, C	One-time onboarding code (stored hashed)
GET, POST /admin/resources; PUT, DELETE /admin/resources/{id}	A	Manage resources and emergency contacts
GET, PUT /admin/engine-config	GET A and C; PUT A	Read or update engine thresholds
GET /admin/audit-logs; GET /admin/audit-actions	A	Paged audit log and list of action names
GET /admin/deletion-requests; DELETE /admin/beneficiaries/{id}	A	Erasure workflow
POST /admin/retention/run	A	Purge old free text
GET /resources	any	Active resources
GET, POST /me/consents	B	Consent state and changes
POST /me/deletion-request; GET /me/export	B	Request erasure; download own data
GET /health	public	Database and model status

Appendix B: Engine configuration defaults

These values are the defaults in the engine configuration; an administrator can change them (subject to validation) and each stored result keeps the configuration it was scored with.

Setting	Default	Meaning
attention_overall	40	Overall index at or above this gives Attention
urgent_overall	70	Overall index at or above this gives Urgent review
attention_dimension	65	Any single dimension at or above this gives Attention
urgent_dimension_safety	75	Safety dimension at or above this gives Urgent review
rise_attention / rise_urgent	15 / 30	Rise in overall index since the previous check-in
dimension_rise_attention	25	Rise in the sleep or daily-activities dimension
persistence_dimension / window / min count	50 / 3 / 2	A dimension at or above 50 in at least 2 of the last 3 check-ins
min_completion	0.6	Share of closed-ended items needed before threshold rules apply
self_report_worse_concern	3	Concern value of the change item at which Attention is raised
critical_self_harm_item	on	Any non-zero answer gives Urgent review
critical_unsafe_concern	3	Concern value of the two safety items at which Urgent review is raised
help_attention_concern / help_urgent_concern	3 / 4	Contact wished within days / as soon as possible
assessment_interval_days	14	A new check-in is shown as due after this many days
dimension_weights	safety 1.5, others 1.0	Weights in the overall index

Other defaults: access token lifetime 60 minutes; account lock-out after 5 failed logins for 15 minutes; bcrypt cost 12; analytics minimum cell size 3; free-text retention 365 days.

Appendix C: Running the project and demo accounts

C.1 Static prototype (no backend)

Open the prototype and choose **Beneficiary**, **Caseworker** or **Admin** on the sign-in page. Alternatively, sign in with a sample username and the password `Demo-pass-1`:

Username	Role
admin	Administrator
caseworker1 to caseworker5	Caseworkers (five assigned cases each in the sample data)
ben001 to ben025	Beneficiaries (ben001 is assigned to caseworker1)

The password applies only to the synthetic sample accounts in the static prototype. Use **Reset demo data** in the banner to restore the original sample data.

C.2 Full stack

The full backend does not contain any default passwords. After copying `.env.example` to `.env`, the seed script creates the same usernames with the passwords you put in the three `SEED_*_PASSWORD` variables. The commands are listed in Section 11.2 and in the repository `README.md`.

C.3 Documents in the repository

File	Content
README.md	Overview, installation, running, tests, limitations
DEPLOY.md	Step-by-step hosting guide
docs/system_architecture.md	Architecture, data flow, workflow, design decisions
docs/database_design.md	Tables, constraints, ER diagram, retention
docs/api_documentation.md	Endpoint reference
docs/assessment_methodology.md	Questionnaire, scores, rules, missing data
docs/ml_methodology.md	Synthetic data, model, metrics, limits
docs/security_and_privacy.md	Controls and residual risks
docs/testing_strategy.md	Test layers and recorded results
docs/reproducibility.md, docs/future_scope.md, docs/implementation_checklist.md	Rebuild steps, future work, status checklist